

University of Debrecen

Faculty of Science and Technology

Institute of Physics

Department of Electrical and Electronic Engineering

Electrical BSc specialty

Design and Development of an Integrated Water and Air Quality Monitoring Sensor Data Collecting System

Soliman Rahma, FO3C4A

Supervisor: Lajos Harasztosi

ABSTRACT

This thesis covers the creation of an entire environmental monitoring system using the ESP32 microcontroller to measure air and water quality using a combination of multiple sensors. For gas detection, the system combines available sensors like the MQ-2 and MQ-135; for temperature and humidity, DHT11; for water temperature, DS18B20; for dissolved solids, TDS; and a turbidity sensor to determine water quality. Before moving to a purchased sensor for more consistent results, a customized pH sensor circuit was also designed and tested. For real-time monitoring and analysis, the data from every sensor flows to the Thing-Speak platform.

ACKNOWLEDGEMENTS

I am very thankful to all those who provided me with assistance and encouragement throughout my thesis writing. First, I would like to express my appreciation to Professor Harasztosi Lajos, my supervisor. He provided me with guidance and support throughout all stages providing me with knowledge and advice. His comments and support helped me achieve my main goals and resolve any challenges that appeared throughout the process.

Also, I would want to express my appreciation to my friends, family, and fiancé for their constant encouragement and continued trust in me at the moments when I did not believe in myself. As a result of their consistent support, I was able to finish this thesis with confidence, and I would not have been able to achieve success without them.

TABLE OF CONTENTS

1. Introduction	6
1.1. Background and Context.....	6
1.2. Objectives and aims	8
2. Literature Review	9
2.1. Water quality standards.....	9
2.2. Air quality standards	10
2.3. Internet of things implementation	11
2.4. Previous conducted research	13
3. System components.....	15
3.1. Block diagram	16
3.2. Esp32.....	17
3.3. The Sensors	19
3.4. Testing on Breadboard	28
4. System Implementation	30
4.1. pH Sensor Design	30
4.2. Soldering and Testing Signal	32
4.3. pH Sensor Circuit TINA	34
4.4. Entire System Circuit	35
4.5. Soldering components for Entire System.....	37
4.6. Error Analysis	38
4.7. PCB with sensors	39
5. Results and Data	40
5.1. MQ2 Data.....	40
5.2. MQ135 Data.....	42
5.3. DHT11 Data	43
5.4. DS18 Data	44
5.5. Turbidity Data	45
5.6. TDS Data.....	46
5.7. pH Data	47
6. Conclusion	48
7. References.....	49
8. Appendices	51

ABBREVIATIONS

ADC Analog to Digital Converter

DAC Digital to Analog Converter

PWM Pulse Width Modulation

SPI Serial Peripheral Interface

I²C Inter-Integrated Circuit

SDA Serial Data Line (I²C Communication)

SCL Serial Clock Line (I²C Communication)

UART Universal Asynchronous Receiver-Transmitter

pH Potential of Hydrogen

TDS Total Dissolved Solids

MQ Metal Oxide Semiconductor (MQ-series Gas Sensors)

CO Carbon Monoxide

CO₂ Carbon Dioxide

NH₃ Ammonia

H₂ Hydrogen

NO₂ Nitrogen Dioxide

DHT Digital Humidity and Temperature Sensor

MCU Microcontroller Unit

PCB Printed Circuit Board

OLED Organic Light Emitting Diode

IoT Internet of Things

GND Ground

VCC Voltage Common Collector (Positive Supply Voltage)

1. Introduction

Precise and constant monitoring of water and air quality has become a major concern among rising pollution levels in protecting our natural environment. The pressure on these resources increases as the world's population grows and industrial activity advances, therefore stressing the urgent need for complete environmental management plans.

The goal of this thesis is to help us better understand and keep track of the different effects of environmental pollutants in water and air systems. The objective of the project is to show how to monitor sensors and why real-time data collection and analysis are important by using better sensor technology and the Internet of Things (IoT).

1.2. Background and Context

A big problem is that the quality of the water and air is getting worse, which hurts ecosystems, people's health, and the safety of economies all over the world. The need for these important resources grows along with cities and businesses. This makes it even more important to have monitoring systems that are thorough and work well. Taking regular samples and analyzing them in a lab is a traditional way to keep an eye on the environment. These ways work, but they miss changes that happen in real time and might be too expensive to use on a large scale.

Improvement in sensor technology and digital networking is making it possible to solve these problems in big ways. New sensors can give constant, real-time information on a lot of different environmental factors. You can connect these sensors to the Internet of Things (IoT) and they can send data right away to systems that are online. This lets systems quickly look at things and act on them. Being able to do this is very helpful for adapting to changing conditions in the environment.

Different kinds of sensors are going to be used to create a monitoring system for this project to give full information on the quality of both the air and the water. For water analysis, a TDS sensor measures the number of dissolved particles, which gives information about how pure the water is overall. A turbidity sensor was added to measure the clarity of the water. These two sensors work together to give a better picture of the water quality and possible pollutants. The DS18B20 temperature sensor was also used to keep an eye on the water temperature and make sure that any changes were recorded correctly. Lastly, a home-made pH sensor will be developed and tested out.

For air quality, the system will use MQ-2 and MQ-135 gas sensors. Whereas the MQ-135 is sensitive to ammonia, nitrogen oxides, and carbon dioxide, the MQ-2 senses combustible gases including hydrogen and carbon monoxide. Combining two gas sensors helps the system to efficiently monitor a wider spectrum of air contaminants. Tracking ambient temperature and humidity by the DHT11 sensor improves the arrangement even more by giving information that might affect gas sensor readings. The ESP32 microcontroller handles all sensor data and sends it to the ThingSpeak platform for remote monitoring and analysis so allowing real-time environmental condition data logging.

1.3. Objectives and Aims

My main goal is to create a network of sensors that work together to accurately and continuously check the state of the air and water. The goal of this project is to make an environmental monitoring system that is complete, scalable, and affordable by combining the benefits of available sensors with the custom sensor into IoT technology.

Key Objectives of the thesis:

- Initial Sensor Testing After Purchase: Breadboard prototyping starts with testing commercially purchased sensors (TDS, DS18B20 temperature, MQ-2, MQ-135, and DHT11 sensors) on a breadboard. During this first stage, I will be able to evaluate the performance and data accuracy of every sensor.
- Custom Sensor Fabrication: Design and create a sensor meant specifically for pH measurement. The sensor will be built with great sensitivity and accuracy required to detect changes in water quality environment.
- IoT Connectivity: Connect system to an IoT network utilizing technology to provide smooth data transfer, storage, and analysis from the integrated sensor network. This digital technology will enable real-time monitoring and remote access to environmental data.
- PCB Design and Assembly: Convert the completely checked and optimized breadboard-based sensor network into a printed circuit board (PCB). The PCB layout is created, components are soldered, and the final product is built.
- System Review and Fault Analysis: Examine the PCB-based system closely to identify and correct any faults or mistakes. This last evaluation ensures that the system is dependable, steady, and ready for practical use.

2. Literature Review

As pollution from more factories and cities grows and causes more environmental health problems around the world, advanced monitoring systems become more important. Ideally these devices must work well to collect real-time data on air and water quality, but they also need to be affordable for general use. As sensors, the Internet of Things (IoT), and technology get better, smart systems for monitoring water and air pollution become more important. This chapter gives an overview of the latest progress made in smart systems for monitoring air and water pollution, as well as how the Internet of Things (IoT) is being used.

2.1. Water Quality Standards

The World Health Organization (WHO) asserts that clean and safe water is necessary for consumption, domestic use, industrial applications, and health. Contaminated water and insufficient sanitation may promote the transmission of diseases such as cholera, diarrhea, hepatitis, skin infections, typhoid, and other health issues. Approximately 2 billion individuals globally lack access to safe water and feel obligated to use contaminated water. [1]

Water quality monitoring involves collecting data at certain locations and durations to describe current water conditions. Smart water quality monitoring systems have the following objectives: measuring critical quality indicators like physical, chemical, bacterial qualities, and analyzing sensor data in real time offering appropriate revision steps. [1]

Table 1 describes the primary water quality standards that are crucial for maintaining and protecting aquatic environments. Those standards are more than simply figures; they are critical benchmarks for environmental monitoring that guide initiatives to satisfy legal requirements and efficiently evaluate ecological condition. Understanding the condition of water bodies depends critically on the given values: temperature, turbidity, total dissolved solids (TDS), and pH. In industrialized areas, where the well-being of local species and communities directly depends on the quality of the water, this knowledge is extremely important. Following these rules helps us to ensure that water quality stays within safe limits, therefore stressing the need for continuous environmental monitoring in industrial environments. [2]

Parameters	Standard Value
Temperature (°C)	25 - 37
Turbidity (NTU)	≤ 5 NTU
Total Dissolved Solids (ppm)	170 - 360
pH	6.5 - 8.5

Table 1: Water quality standard values [2]

2.2. Air Quality Standards

Traditional methods, such as lab analysis and pricey models, are no longer useful. Recent research focuses on smart devices using machine learning techniques, big data technologies, and the Internet of Things to gather and analyze air data. [3]

With approximately 4.1 million fatal accidents worldwide in 2019, including 500,000 in Europe and 36,000 in Poland alone, the World Health Organization (WHO) offers depressing estimates on the major effects of outdoor air pollution. These numbers show how urgently strong pollution control policies and efficient air quality management are needed to preserve human health. Over enormous portions of the atmosphere, pollutants including carbon monoxide (CO), ozone (O₃), nitrogen oxides (NO_x), sulfur dioxide (SO₂), and spread particulate matter. Public health as well as world air quality are greatly affected by this kind of pollution. Monitoring these dangerous chemicals produced by surrounding industries helps one to compare high pollution rates with reference levels and determine when the quality falls below at some point, thereby alerting people to say that something is not safe. [4]

The World Health Organization's general air quality criteria—used as benchmarks for worldwide air quality monitoring are assembled in Table 2. These rules aim to protect public health and the surroundings from the bad effects of air pollution. The WHO wants to help national and worldwide air quality management operations by setting limits on important pollutants including carbon monoxide, nitrogen dioxide, ozone, sulfur dioxide, and particulate matter. [5]

Pollutant	Averaging Period	WHO Standard
Carbon Monoxide (CO)	15 minutes	100 mg/m ³
	1 hour	35 mg/m ³
	8 hours	10 mg/m ³
Nitrogen Dioxide (NO ₂)	1 hour	200 µg/m ³
	24 hours	25 µg/m ³
	Annual	10 µg/m ³
CO ₂ (Carbon Dioxide)	8 hours	100 µg/m ³
	8 hours	Less than 1,000 ppm
Temperature		20-24°C (68-75°F) 23-26°C (73-79°F)
Humidity		30-50% RH (Relative Humidity)

Table 2: Air Quality Standards [5]

2.3. Internet Of Things Implementation

The Internet of Things (IoT) presents an opportunity to improve environmental pollution and water monitoring. It enables immediate data finding, interpretation, and sharing. Because of their ability to overcome the gap, affordable IoT-based environmental monitoring devices have caught the curiosity of many. Devices that measure air and water quality are cost-effective and accessible to a wider user base. Low-cost pollution monitoring systems are intriguing alternatives to traditional monitoring systems due to their affordability and scalability, especially in resource-constrained contexts. [6]

The Internet of Things (IoT) refers to the widespread use of cyber-physical systems with increased sensing, communication, and capabilities. The Internet of Things (IoT) is a network of connected devices with sensors and actuators that allow data to be collected, analyzed, and shared across the internet. All IoT devices, such as smartphones and tablets, operate in the end device layer. Users who want to know about air and water quality can connect their devices and access shared information. Users should set up a client application on their devices. The client application allows you to log into the server and obtain air quality data. [4]

Despite the potential benefits of IoT-based water and air quality monitoring systems, numerous problems and considerations must be addressed throughout installation. These include power supply limits for remote sensor nodes, communication challenges in remote or subsurface locations, concerns about data security and privacy, and regulatory compliance with quality standards. Figure 1 illustrates IoT monitoring. [7]

The use of IoT-based real-time water and air quality monitoring systems also has various advantages. For starters, it provides early detection of anomalies or contamination incidents, allowing for timely reaction steps to reduce public health hazards. Second, constant monitoring enhances operating efficiency by optimizing chemical dosage, lowering energy use, and decreasing waste. Furthermore, the availability of complete data sets enables trend analysis, predictive maintenance, and informed decision-making for long-term infrastructure planning. [7]



Figure 1: IoT Monitoring [7]

2.4. Previously Conducted Research

The Central Water Commission (CWC) monitors water quality by obtaining samples from designated locations across the processing and distribution system. These samples are tested at the adequately equipped facilities. These laboratories gather samples of raw water, filtered water, and treated water for examination. Meters are employed to assess water characteristics, including turbidity, pH, and dissolved oxygen, among others. The drawbacks of the existing methodology include the absence of continuous and remote monitoring, the necessity for human resources, diminished reliability, and a lack of oversight at the water source. Given the constraints of the existing system, it is imperative to develop a system that facilitates real-time and continuous monitoring of water quality. Numerous advanced techniques for monitoring water quality have been introduced in recent years. A configuration for a wireless sensor network featuring several sensor nodes situated in a lake is proposed. A significantly decreased quantity of UAVs surveils the lake and is overseen by the central monitoring station (CMS). The CMS gathers and analyzes sensor data. A framework for monitoring water quality is established using a wireless sensor network, which includes the assessment of bacterial contamination in open water bodies. [9]

Hendrawati et al. did research on the Water Quality Monitoring System with the goal of monitoring industrial-related pollution in river water. Sensor nodes provide three parameters for measuring water quality: PH, turbidity, and temperature. The primary node will receive data from the sensor node via the nRF24L01 radio communication module. The data is sent to the server via ESP-8266, and users can access it through web and android apps. [10]

Waleed et al. undertook similar studies to inform the community about river water quality. Sensors measure pH level, turbidity, and water temperature. Sensor input data is converted into usable values and transmitted to the server via Long Range Network (LoRa). The server uses the Fuzzy Logic Method to classify data into three statuses: low, medium, and high, which are displayed on user-accessible websites. This study improves categorization results by utilizing a TDS sensor, distinguishing it from the previous one. [10]

Kumar Sai et al. suggested an affordable IoT-based air quality monitoring system using Arduino and the MQ series (MQ135 and MQ7). These sensors can detect ammonia, carbon dioxide, alcohol, smoke, and carbon monoxide. This configuration provides a cost-effective and flexible way to show air pollutants. The system assesses air quality using data from

several sensors, demonstrating the effectiveness of low-cost sensors for environmental monitoring. This technique increases air quality monitoring and uses IoT to address environmental concerns. The authors underline the need for accessible air quality monitoring to promote public health. [3]

Zhao et al. built an air quality meter by attaching sensor nodes to Unmanned Aircraft vehicles using LoRa technology. UAVs successfully collected information from sensor devices, covering approximately 200 m in 118 seconds. A UAV-based air quality monitoring system was implemented, achieving two objectives. It might operate in a real-time framework and conduct sensing duties. Researchers aim to implement novel sensor node approaches that can measure parameters and transmit data to the cloud via LoRa. Air quality values of CO₂ and PM_{2.5} were successfully monitored in the University of Rwanda cafeteria kitchen and laboratory. [11]

A scalable system for monitoring air quality in Myanmar was presented, utilizing low-cost sensor nodes and the LoRa-WAN protocol for long-range communication. Sensor nodes in Yangon capture and transmit real-time data on air factors like dust, humidity, temperature, and carbon dioxide to monitor and predict air pollution in the region. Users can view data on the Grafana dashboard using their Google login credentials to access the server. The data indicates a temperature range of 26-32°C, humidity range of 79-100% H, dust range of 1-70, and CO₂ range of 406-529 ppm. [11]

3. System Components

The ESP32 microcontroller is the main control board for my environmental monitoring project. The ESP32 is perfect for this kind of project because it makes it easy to connect both digital and analog environmental sensors. The ESP32 also comes with Wi-Fi and Bluetooth, which makes it easy for me to send data wirelessly and keep an eye on the environment from afar and in real time.

Choosing the right sensors is important for keeping track of a wide range of air quality indicators. The MQ-2 gas sensor, which can find gases that can catch fire, such as propane and methane. The MQ-135 sensor, which works with the MQ-2, greatly increases our ability to find important contaminants like ammonia and nitrogen oxides. These gases are important signs of environmental health because they are released by many things, like car exhaust and manufacturing processes. This lets us find and respond more accurately to changes in pollution levels.

The DHT11 sensor measures both temperature and humidity using a calibrated digital signal output to keep an eye on the weather. The DS18B20 temperature sensor is used for water. The one-wire interface makes wiring easier for long-distance sensing, and the water-proof design and high accuracy make it perfect for profiling water temperatures.

The system has a small OLED screen that shows data being collected in real time. The display shows current readings from all sensors using the SPI interface for fast data transfer. This makes the system more interactive and allows for quick research on-site. A 5V USB power supply powers the whole thing. Because it makes the system more portable, this power source makes it possible to use it in more places than with traditional power sources.

3.1. Block diagram

Figure 2 illustrates the block diagram, which shows how my environmental monitoring system functions together as an entire system. This diagram gives you a general idea of how the different hardware parts are put together and how they communicate with each other. At the center of the diagram is the ESP32 microcontroller. The sensors are grouped together on the left side of the diagram to make their role as data sources for the system stand out. The lines show the data flow from the sensors to the microcontroller. Each sensor is connected to the ESP32 separately. This setup makes it easy to gather information from both water and air, and it also visually separates the system's sensing part from the rest of its design.

The data output and connections are shown on the right side of the image. The OLED display shows local, real-time feedback straight from the ESP32, and the wireless connection to the cloud platform (ThingSpeak), which lets the data be logged and monitored. The power supply block at the bottom shows that all the parts share a power source, which keeps the system stable. Making this block layout helped me get a better idea of how my system would work before I put it together physically. Not only did it help me with wiring and pin mapping, but it also helped me see how each part fits into the whole data flow, from getting the data to handling it to showing it and talking to other devices. This step was important for making a clean plan for both the hardware and software parts of the project.

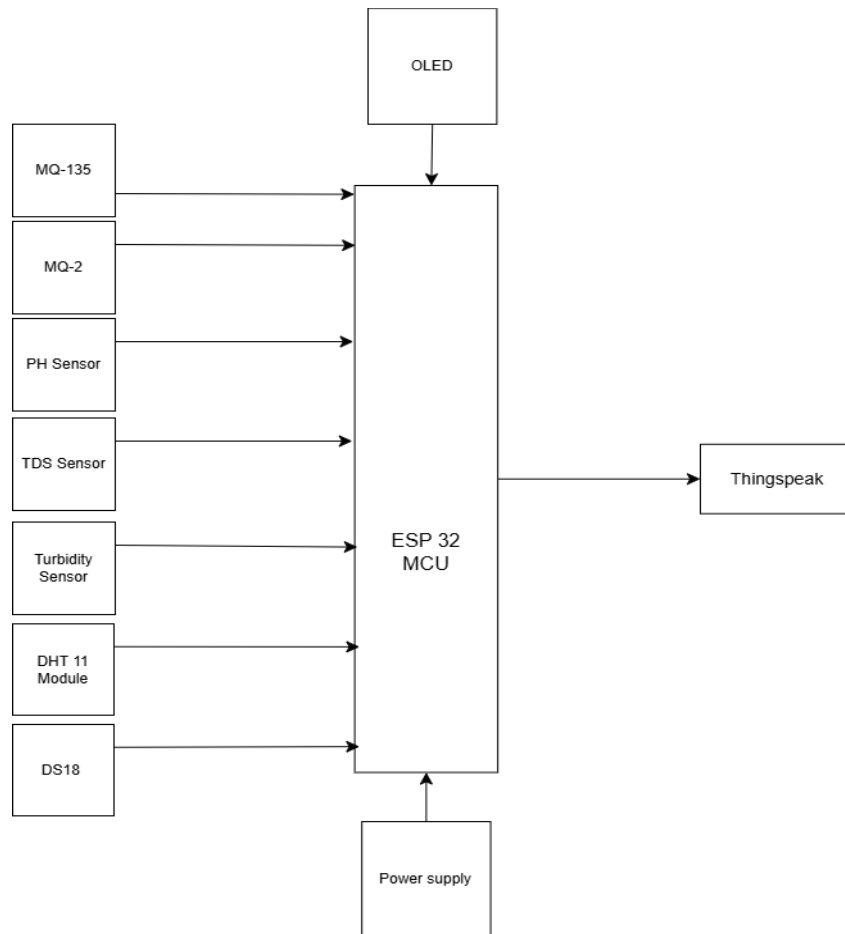


Figure 2: Block Diagram of the system

3.2. ESP 32

Many embedded and Internet of Things (IoT) applications today use the ESP32 micro-controller because it is strong and flexible. Both of the Xtensa 32-bit processors inside can run up to 240 MHz, which means they can do a lot of things at once, like reading sensors, updating a screen, and managing real-time wireless data communication. The fact that the ESP32 had built-in Wi-Fi and Bluetooth is ideal for this project. Because of these features, it works great for Internet of Things (IoT) systems that need to send data to the internet or mobile apps over wireless networks. My current setup is mostly for local monitoring, but the ESP32 lets me make the system bigger in the future so it can be a fully connected IoT solution.

ESP32 has 34 GPIO pins, and some of them can do things like Analog-to-Digital Conversion (ADC), Digital-to-Analog Conversion (DAC), Pulse Width Modulation (PWM), and digital communication protocols like SPI, I²C, and UART. This makes it possible for a wide range of analog and digital sensors to work together smoothly with the system. These include the DHT11, DS18B20, TDS sensor, and gas detectors like MQ-2 and MQ-135. The ESP32 works with 3.3V logic, so any sensors or modules that are connected must either work with 3.3V or have a voltage level shifter to keep them from breaking.

Power can be sent through a micro-USB cable, which also makes code uploading and serial communication easier, or an external 5V power supply can be used if the device is to be used by itself. The USB-to-serial converter that comes with the board is helpful for development because it makes programming and debugging easier with the Arduino IDE. The board has reset and boot buttons that let you manually upload firmware or do a recovery if you need to. The board has flash memory built right into it, which keeps the program and lets you log data permanently if you need to.

Figure 3 illustrates the ESP32 development board, emphasizing key components including the CPU module, GPIO headers, USB interface, integrated voltage regulator, and the compact ceramic antenna utilized for Wi-Fi and Bluetooth connectivity.

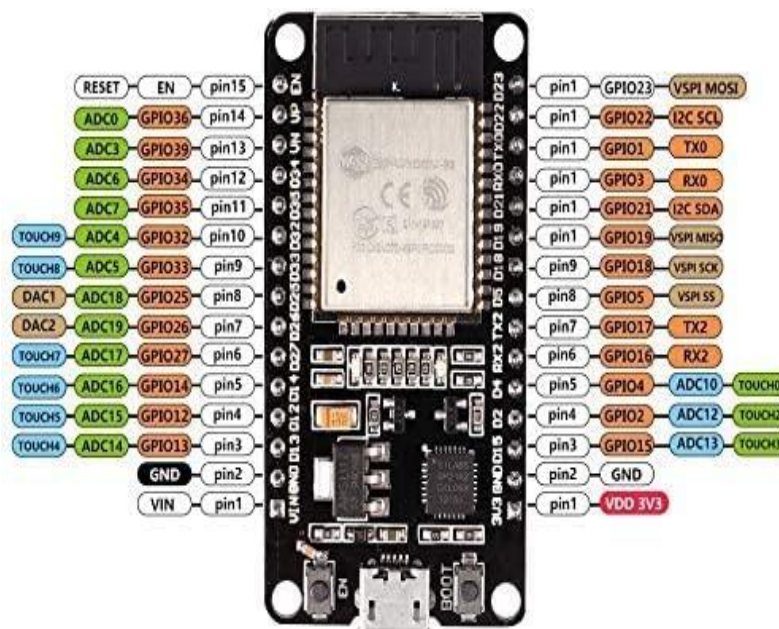


Figure 3: ESP32 Pinout

3.3. The Sensor's

The DHT11 sensor is a convenient and inexpensive component that I used to measure both temperature and humidity in my environmental monitoring project. It generates digital signals, making it simple to connect and communicate directly with microcontrollers such as my ESP32. This sensor has an accuracy of $\pm 2^{\circ}\text{C}$ and can measure temperatures from 0 to 50 degrees Celsius, making it suitable for monitoring environmental conditions. It can measure relative humidity levels ranging from 20% to 80% with a usual accuracy of $\pm 5\%$.

One of my preferred things about DHT11 is how simple it is to use—it only requires three connections: power (VCC), ground (GND), and a single digital data pin. It operates at a standard voltage of 3.3 to 5 volts, making it completely compatible with ESP32's operational voltage. Furthermore, it draws very little current, often less than 1.5 mA, making it ideal for battery-powered systems or low-power projects. The sensor communicates via a unique single-wire digital protocol, which means that no complex circuitry is required to interpret its data. The sensor delivers temperature and humidity readings every two seconds, ensuring that I always have up-to-date data. Its basic design and small size also allow me to integrate it seamlessly. Figure 4 clearly illustrates the pin layout of the DHT11 module, while Table 1 gives the necessary specifications of the sensor.

Specification	Details
Operating Voltage	3.3V - 5V DC
Measurement Range (Temperature)	0°C to 50°C
Temperature Accuracy	$\pm 2^{\circ}\text{C}$
Measurement Range (Humidity)	20% to 80% RH
Humidity Accuracy	$\pm 5\%$ RH
Interface Protocol	Single-wire digital
Number of Pins	3 Pins (VCC, GND, DATA)
Compatible Microcontrollers	Arduino, ESP32, ESP8266, Raspberry Pi, etc.

Table 1: DHT11 Specifications

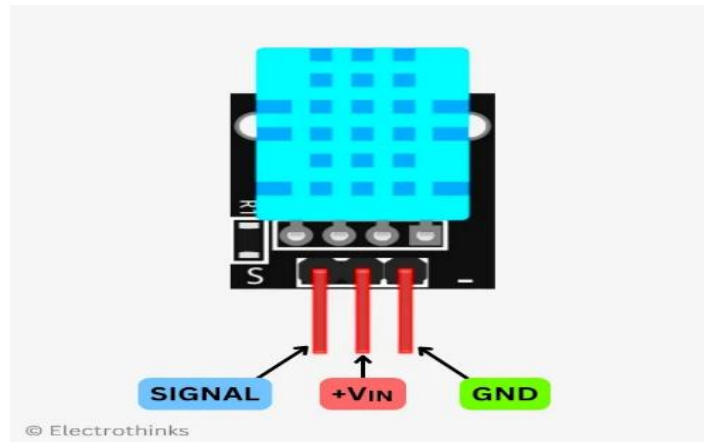


Figure 4: DHT 11 Pinout

The DS18B20 digital temperature sensor is accurate and flexible. It will be used to measure water temperature accurately for the environmental monitoring project. Unlike many other monitors, this one gives you digital temperature data right away, which makes it much easier to collect data. A simple and useful "1-Wire" interface lets it talk to the main microcontroller. This means that data can be sent using just one digital line, along with ground. This makes connecting a lot easier and makes the system less complicated.

This sensor can correctly measure temperatures across an amazing wide range, from -55°C to +125°C. This range covers almost all the temperature ranges that are needed in the real world. The probe is very accurate, within 0.5°C, across the temperature range of -10°C to +85°C. This makes it perfect for keeping an eye on marine ecosystems, hydroponic systems, and other places that are submerged in water. The adjustable resolution of the DS18B20, which ranges from 9 to 12 bits, is one of its best features. It gives steps as small as 0.0625°C at its highest resolution (12-bit), which makes it very useful when accuracy is important. The power range of the DS18B20 is 3.0 to 5.5 volts DC, so it works with the ESP32. Its low standby current (about 1 μ A) makes it perfect for uses that need to use little power or batteries. The DS18B20 often comes in the form of a waterproof stainless-steel probe that works great for submerging it in liquids for a long time. For my water-quality tests, I used this waterproof model. The specifications are given in Table 2, where the pinout is illustrated in Figure 5.

Specification	Details
Operating Voltage	3.0V - 5.5V DC
Temperature Range	-55°C to +125°C
Accuracy	±0.5°C (from -10°C to +85°C)
Resolution	Selectable 9-bit to 12-bit
Communication Protocol	1-Wire digital interface
Typical Current (Active Measurement)	~1 mA
Number of Pins	3 pins (VCC, GND, Data)
Compatible Microcontrollers	ESP32, Arduino, ESP8266, Raspberry Pi
Form Factor	Waterproof (Stainless steel probe).

Table 2: DS18B20 Specifications

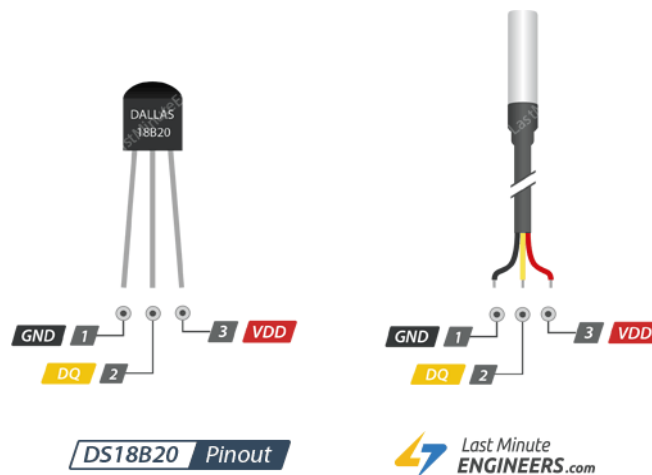


Figure 5: DS18B20 pinout

The MQ-2 gas sensor is one of the most popular and reasonably priced air quality sensors used in electronics applications, and with good reason. I used the MQ-2 to detect multiple flammables and possibly harmful gases, including hydrogen and carbon monoxide. This sensor is highly useful for monitoring locations where gas leaks may represent a safety risk, such as kitchens, garages, or industrial facilities. The MQ-2 works by measuring changes in the resistance of its internal sensing material (a tin dioxide layer), which fluctuates based on the number of gases in the air. As the gas concentration rises, the resistance falls, and the sensor produces an analog voltage proportional to the gas level. This analog signal is easily received using the ESP32's ADC pins, allowing me to track air quality levels in real time.

The sensor module often has both analog and digital outputs, making it versatile: I may obtain a continuous analog reading or set a threshold and receive a digital HIGH/LOW signal when the gas concentration surpasses that level. It is powered by 5V, and while it requires some preheating time (about 20 seconds to a few minutes), it immediately stabilizes and provides consistent readings after that. As you can see from Table 3, the sensors details are provided. Figure 6 illustrates the pinout.

Specification	Details
Operating Voltage	5V DC
Detection Gases	LPG, methane, butane, hydrogen, alcohol, smoke
Concentration Range	300 – 10,000 ppm
Sensor Type	Tin dioxide (SnO ₂) gas-sensitive layer
Analog Output	0–5V (depends on gas concentration)
Preheat Time	20 seconds (recommended: 2–3 minutes)
Response Time	< 10 seconds
Interface	Analog and Digital
Compatible Boards	Arduino, ESP32, ESP8266, Raspberry Pi
Dimensions	~32mm x 22mm

Table 3: MQ2 Specifications



Figure 6: MQ2 Pinout

The MQ-135 gas sensor identifies dangerous chemicals such as nitrogen oxides (NO_x), carbon dioxide (CO_2), and ammonia (NH_3). These gases are frequently present in polluted urban surroundings, industrial locations, and even enclosed indoor spaces with ventilation. The MQ-135 is sensitive to changes in air composition and responds to growing levels of these contaminants by changing its internal resistance.

This adjustment generates a changing analog output voltage, which the ESP32 receives and translates into useful air quality information. This sensor is valuable due to its wide sensitivity and adaptability. For example, nitrogen oxides, which are frequently generated by car exhaust and factories, are markers of outdoor pollution and cause serious respiratory dangers. Ammonia, which is present in cleaning products and released in agricultural settings, can be unpleasant and dangerous at high doses. The MQ-135 is not as exact for CO_2 as more specialized sensors, but it nevertheless functions effectively as an early signal of increasing carbon dioxide levels.

The sensor is simple to use, requiring simply power and one analog connection to the ESP32. It also has a digital output with an integrated potentiometer for setting a gas detection threshold, which might be useful for sending alarms. Although it takes a short warm-up period to stabilize, the MQ-135 provides consistent readings once initialized. Its true strength is in its capacity to provide a broad overview of air quality, making it a practical and cost-effective alternative for monitoring locations where hazardous substances may be present. Table 4 displays the information needed and Figure 7 illustrates the sensor pinout.

Specification	Details
Operating Voltage	5V DC
Detection Gases	Ammonia (NH_3), NO_x , benzene, smoke, CO_2
Concentration Range	10 – 1000 ppm
Sensor Type	Tin dioxide (SnO_2) gas-sensitive layer
Analog Output	0–5V (depends on gas concentration)
Preheat Time	20 seconds (recommended: 2–3 minutes)
Response Time	< 30 seconds
Compatible Boards	Arduino, ESP32, ESP8266, Raspberry Pi

Table 4: MQ135 Specifications

MQ-135 Pinout

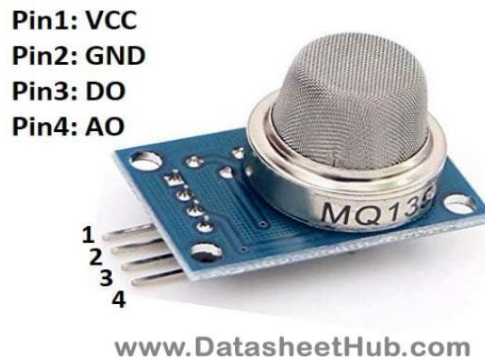


Figure 7: MQ135 Pinout

The TDS sensor—that is, Total Dissolved Solids—is an important part of my water quality monitoring system. It counts dissolved minerals, salts, and organic matter among other elements in water. Natural sources like rocks and soil as well as pollution like agricultural runoff or industrial waste can provide these compounds. One fast and efficient approach to evaluate the quality of the water is monitoring them. This sensor is useful for delicate systems like hydroponics, aquaculture, or even drinking water purification where small variations in water quality can have a significant impact.

The way it easily communicates with the ESP32 is very useful. Its output is an analog voltage depending on the conductivity of the water; more dissolved particles increase the conductivity and thus the voltage. I have a formula in my code that turns this voltage into a readable TDS number expressed as ppm, or parts per million. Figure 8 gives the pinout of the sensor. Then we have Table 5 with the required details of the sensor.

Specifications	Details
Operating Voltage	3.3V–5V
Output Signal	Analog voltage (0–2.3V typically)
Measuring Range	0 – 1000 ppm (typical)
Accuracy	±10% Full Scale
Response Time	< 1 second
Interface	Analog (via ADC)
Operating Temperature	0°C – 40°C
Compatible Boards	Arduino, ESP32, ESP8266, STM32, etc.
Signal Conditioning	Built-in op-amp and filter circuit
Sensor Principle	Electrical conductivity of water

Table 5: TDS Specifications

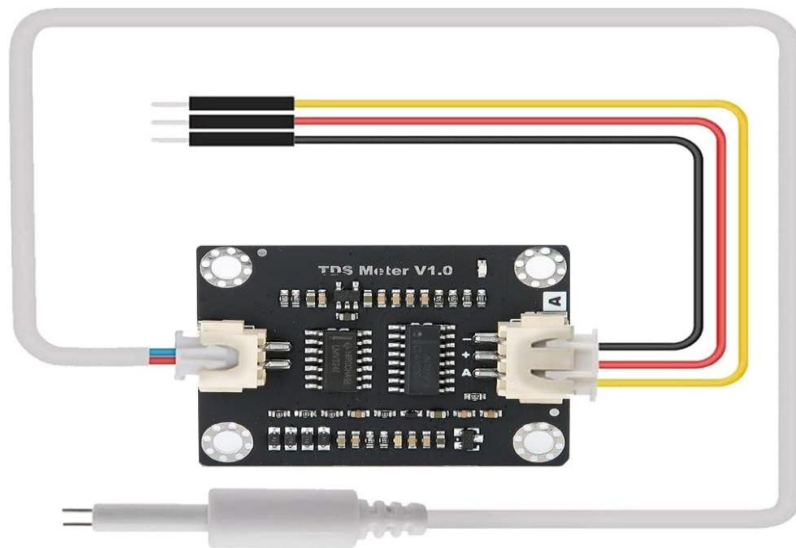


Figure 8: TDS pinout

The turbidity sensor is useful for finding out how cloudy the water is. Turbidity is the presence of particles suspended in water, like silt, algae, and other things that make the water less clear. These very small particles scatter and soak up light, making the water cloudy even though you couldn't see them without a magnifying glass. Think of it like smoke in the air. You can't always see the individual particles, but you can see the haze they make. It is interesting how the sensor works. As it shines a beam across the water, it counts how

much light those particles scatter. Higher turbidity causes more scattering, which means that the water is pretty "dirty," or disturbed. This method works well and quickly, so it's very useful. Turbidity could mean that there are big problems with the environment. High turbidity levels can mean sediment flow, algae blooms, or pollution, all of which are bad for aquatic life because they make it hard for fish and other animals to eat, breathe, or even move.

Figure 9 gives the pinout of the sensor. Then we have Table 6 with the required details of the sensor.

Feature	Specification
Operating Voltage	5V (typically)
Output Type	Analog (AOUT)
Measuring Range	0 – 1000 NTU (Nephelometric Turbidity Units)
Typical Accuracy	$\pm 5\%$ (approximate)
Response Time	< 500ms
Light Source	Infrared LED
Detection Principle	Light scattering (90° detection)
Compatible Microcontrollers	Arduino, ESP32, ESP8266, STM32
Working Environment	Water immersion (non-corrosive fluids)
Calibration Requirement	Yes, for precise NTU accuracy

Table 6: Turbidity specification

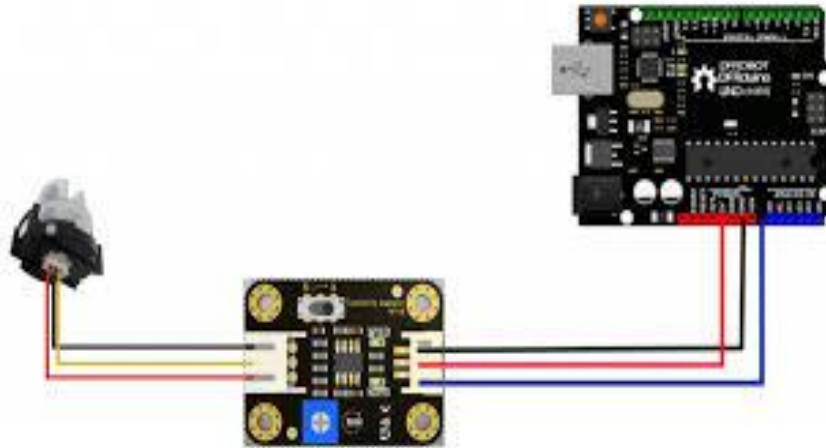


Figure 9: Turbidity Sensor Pinout

The Adafruit's SH1106 OLED screen used in this project offers a clean, sharp visual representation of sensor data. Its 128 by 64-pixel resolution is ideal for displaying numerous lines of text and graphical elements while yet keeping reasonable readability. The performance achieves great contrast and visibility in a range of lighting environments using a basic but powerful monochrome color scheme—white pixels on a black background. Simplifying wiring and reducing complexity, the ESP32 uses the I²C protocol which just requires two data lines—SDA and SCL.



Figure 10: OLED display

3.4. Testing on breadboard

Before moving on to PCB fabrication, I first tested the entire system setup on a breadboard, which gave me the flexibility to make changes and fix wiring easily. During this initial phase, I connected all the sensors to an Arduino Uno, since it's straightforward to use and helped me quickly verify that each sensor was functioning correctly. I tested the gas sensors (MQ-2 and MQ-135), water-related sensors like the TDS, turbidity, and DS18B20, as well as the DHT11 and OLED display. This stage was to make sure I could get readings from all components and verify sensor outputs using the Serial Monitor.

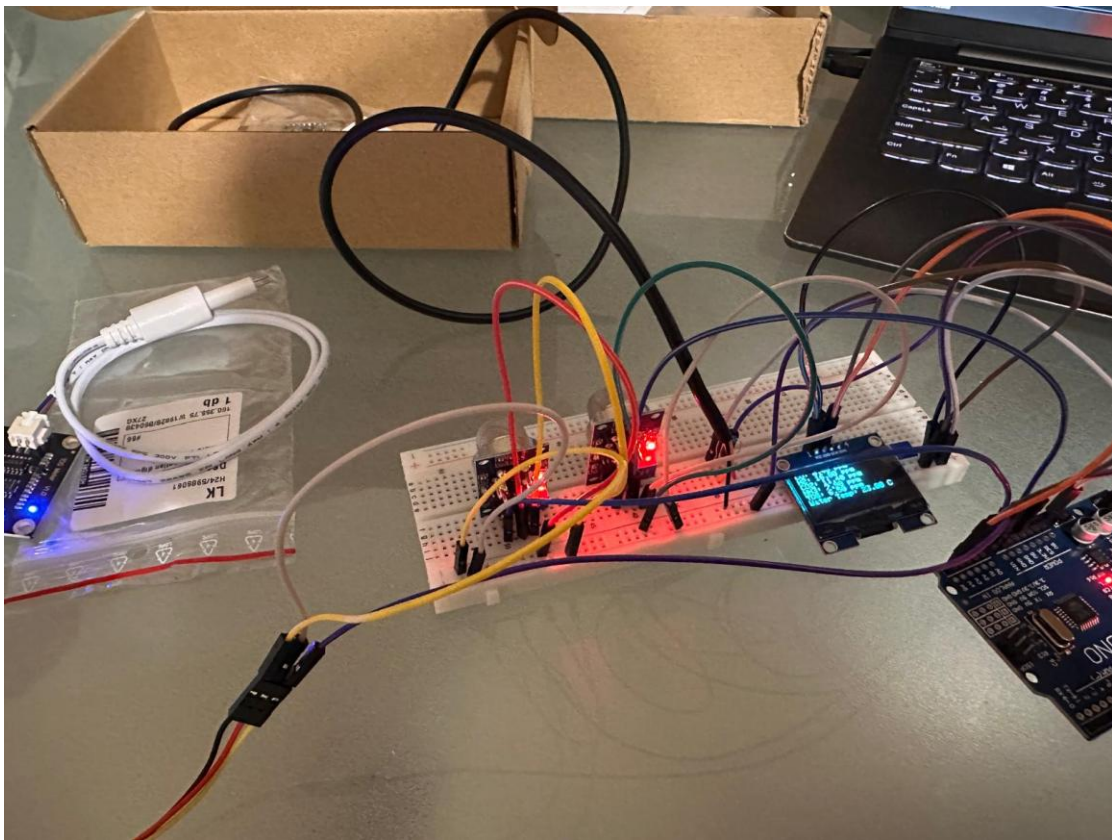


Figure 11: Testing on Breadboard

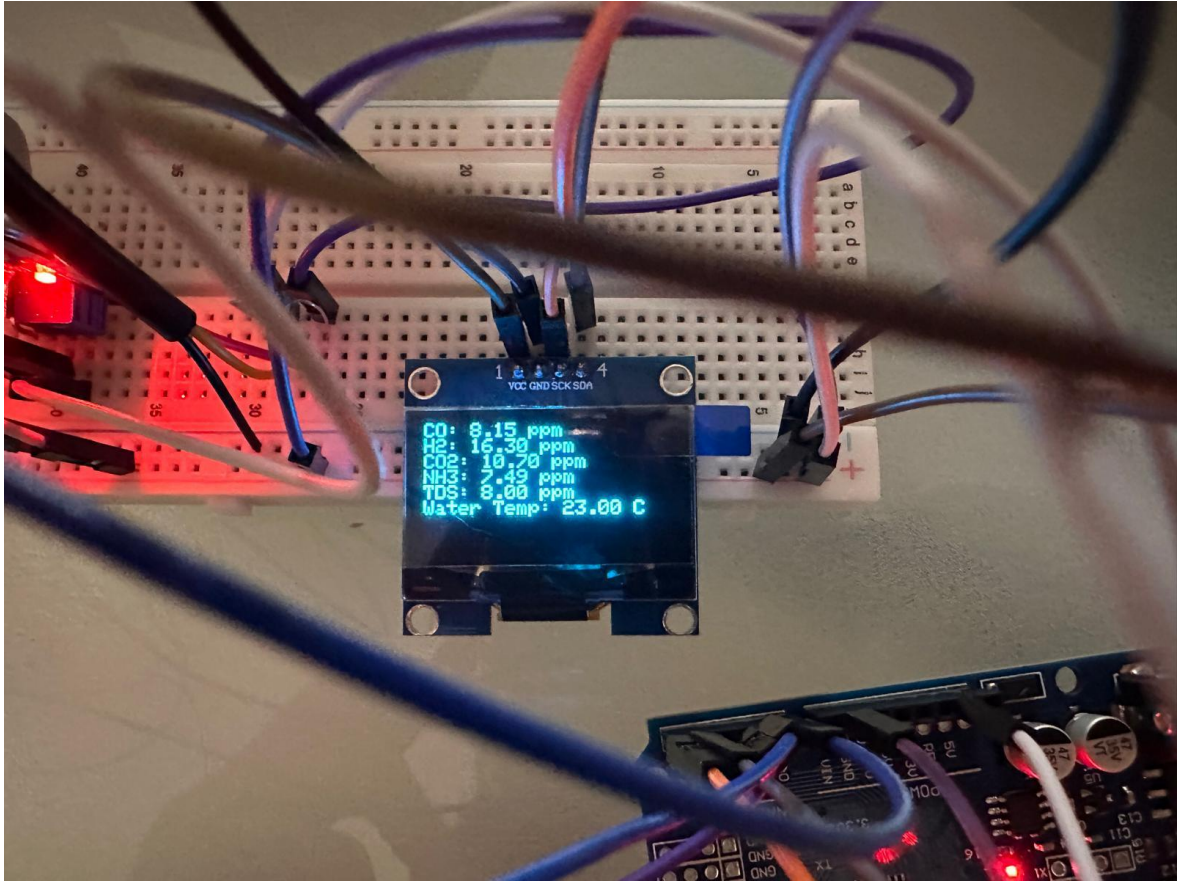


Figure 12: Values on OLED

Following Arduino's basic confirmation, I then turned to the ESP32, which runs on 3.3V logic, so I had to re-test the sensors to ensure they remain compatible and produce precise results.

4. System Implementation

The project relies on schematic design, as it provides a clear and organized visualization of how each sensor and component are connected to the ESP32. To track water and air quality, the schematic assists in the mapping out of how the system is connected all together.

The ESP32's analog input pins connect the TDS sensor, turbidity sensor, pH sensor, and gas sensors (MQ-2 and MQ-135). These sensors provide analog voltages reflecting many environmental variables including acidity, water cleanliness, and gas concentration. On the other hand, the DHT11 sensor and the DS18B20 interact via digital protocols—the DHT11 uses a single-wire digital communication protocol whereas the DS18B20 employs the 1-Wire protocol. These sensors return exact digital data instead of changing voltages; they are connected to digital GPIO pins on the ESP32.

Combining analog and digital sensors into one system helps the designer to manage data from many sources and guarantees consistent and clear signal routes.

4.1. pH Sensor Design

The most practical task in this project was designing the pH sensor circuit. To gain a better understanding of how the signal conditioning operates, I chose to design the circuit from scratch. Reflecting the acidity or alkalinity of the solution it is in; the pH probe generates a small analog voltage. Before the microcontroller determines the signal, though, it must travel through a well-crafted amplification and filtering stage as it is too weak to read directly. This is where the analog circuit design came in; I had to choose capacitors, resistors, and operational amplifiers to create a signal conditioning circuit that would stabilize and scale the probe's signal properly.

I looked over DFRobot's publicly available schematic for their pH sensor module to assist me develop the design process. Their literature provided me with a good basis for component selection and voltage layout, which I modified for my personal arrangement. Figure 13 illustrates the schematic design, while figure 14 illustrates the PCB connections.

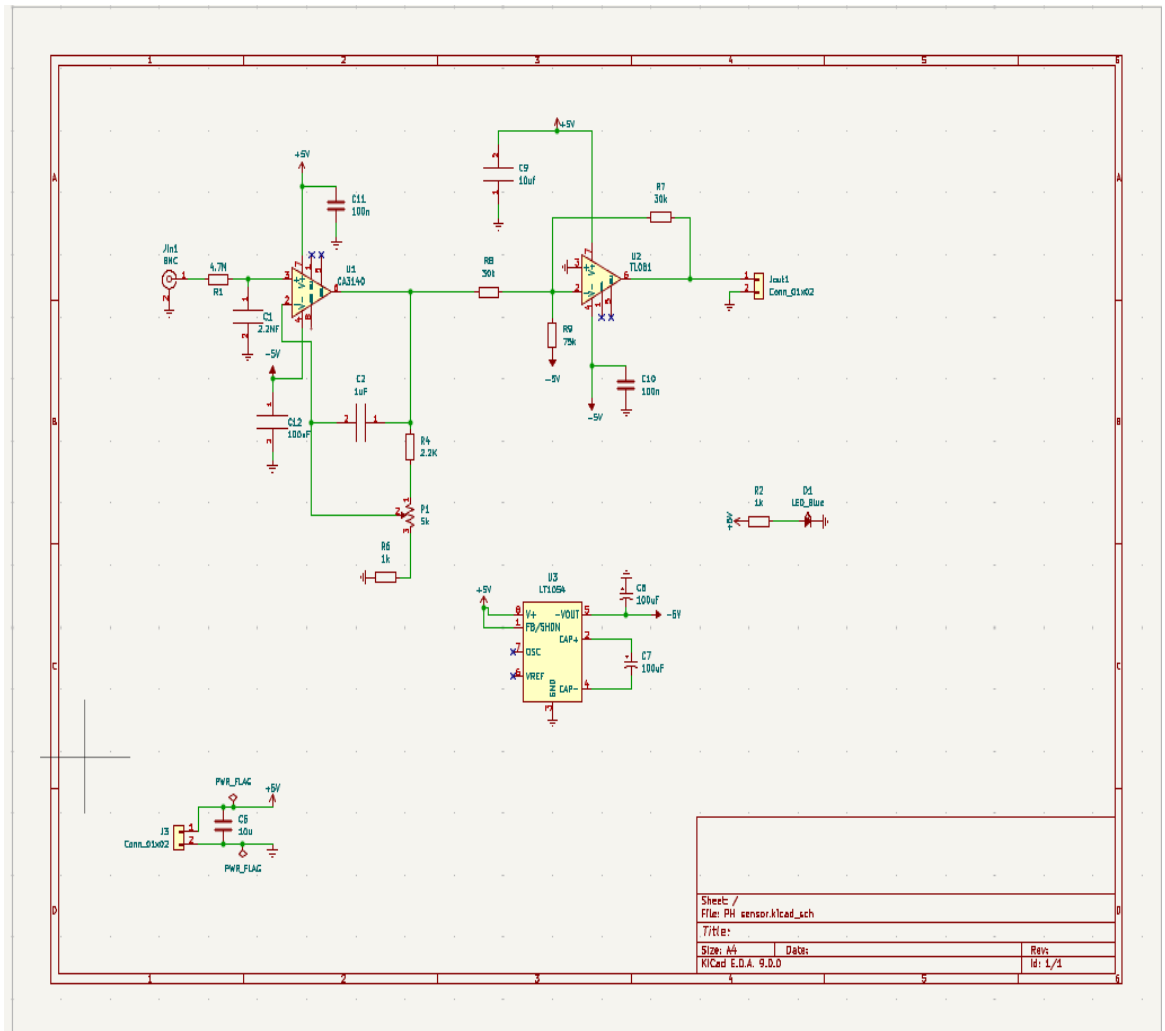


Figure 13: pH circuit schematic

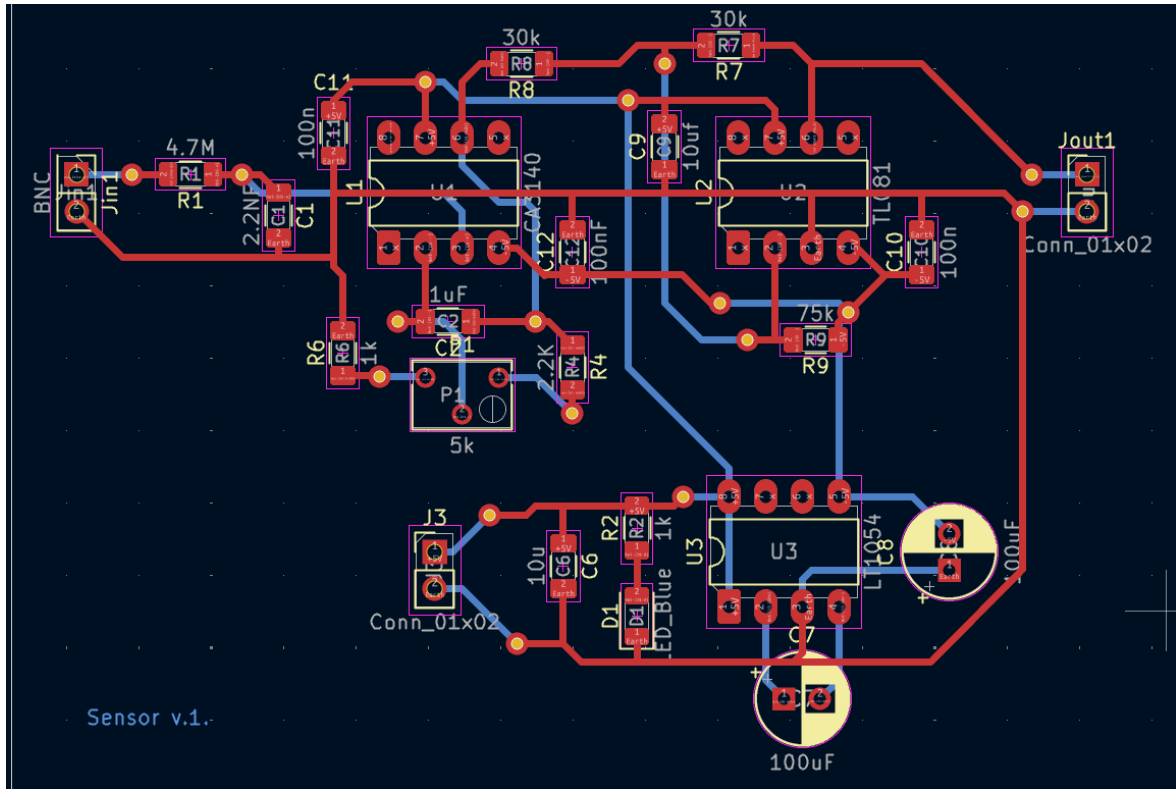


Figure 14: PCB of pH sensor

4.2. Soldering and Testing Signal

To make sure the homemade pH sensor circuit operated correctly, I investigated the operating amplifier input and output voltages. For more accurate data collecting, I decided to use a professional pH sensor instead at the end; however, these experiments were very important to confirm that the custom-built sensor was in fact producing a signal. The oscilloscope figure 16 displays clear waveform signals at the input and output stages of the operational amplifiers.

This demonstrates that the pH signal might be successfully put and processed in the circuit. The waveform signals generated during the pH sensor circuit testing are displayed on the oscilloscope. Different phases of signal processing within the circuit are represented by the three waveforms. The initial, unamplified signal from the pH sensor is shown by the yellow waveform. It is small and weak. The amplitude of the signal increases as it progresses through the initial stage of amplification, as depicted in the green waveform, suggesting that the circuit is successfully increasing the signal's intensity.

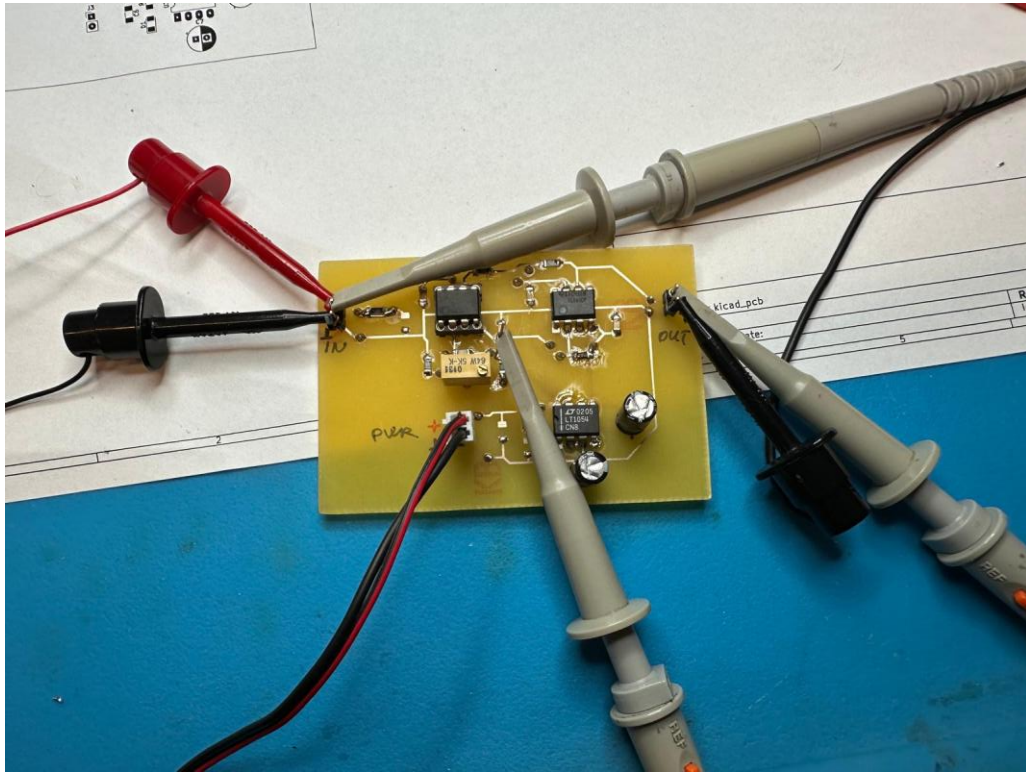


Figure 15: Soldered PH Sensor

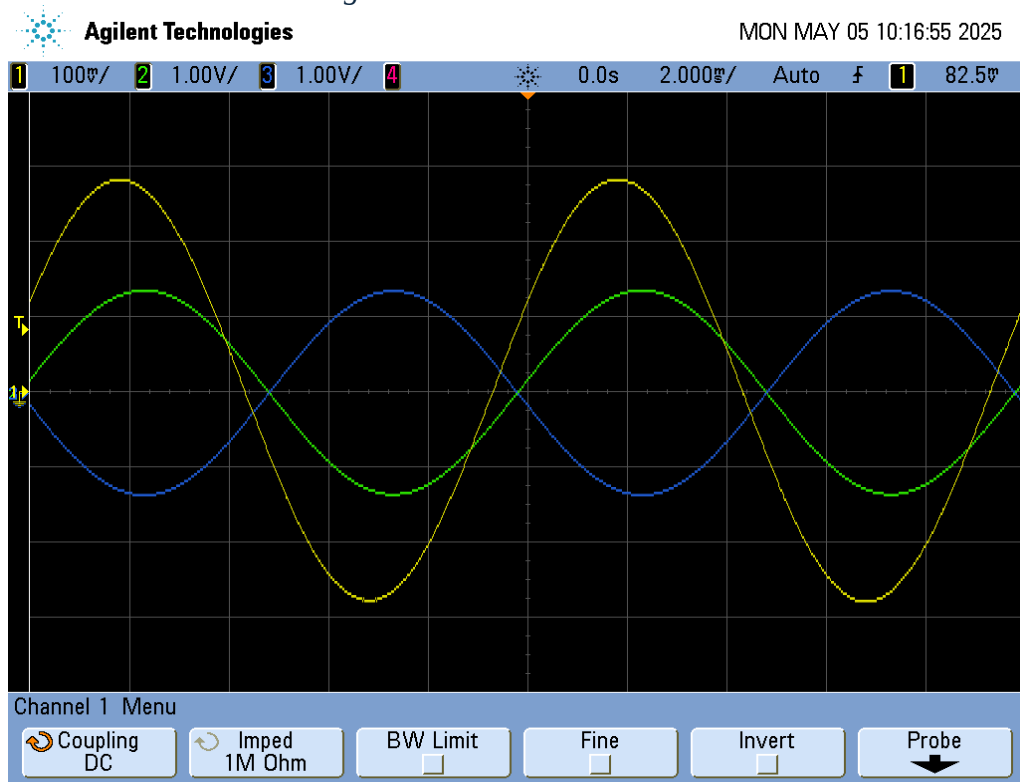


Figure 16: Oscilloscope signals

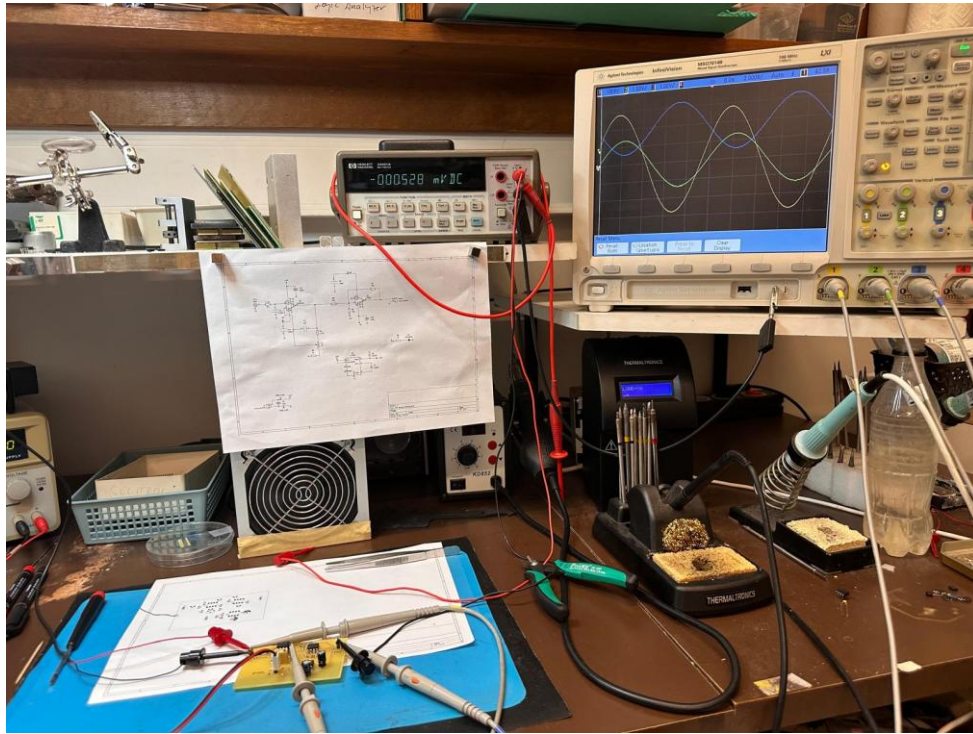


Figure 17: Setup of pH measurements

4.3. PH Sensor Circuit TINA

Operational amplifiers are very important in the amplification step of the pH sensor circuit. They boost the weak pH signal from the probe. Two operational amplifiers are used in a two-stage setup in the circuit. The first op-amp acts as an initial amplifier. It takes the low-voltage signal from the pH probe and boosts it while keeping the signal's purity. The second op-amp boosts the signal even more to a level that can be read. This makes it easier to understand in later stages of data acquisition. During this process, capacitors are placed in a way that keeps the signal stable and lowers the noise, which results in a clean output pattern. The input signal appears on the oscilloscope as a waveform with a lower amplitude. As it moves through each op-amp step, it gains a lot of power. This controlled amplification is needed to properly pick up changes in the pH signal and turn them into a form that the microcontroller can understand.

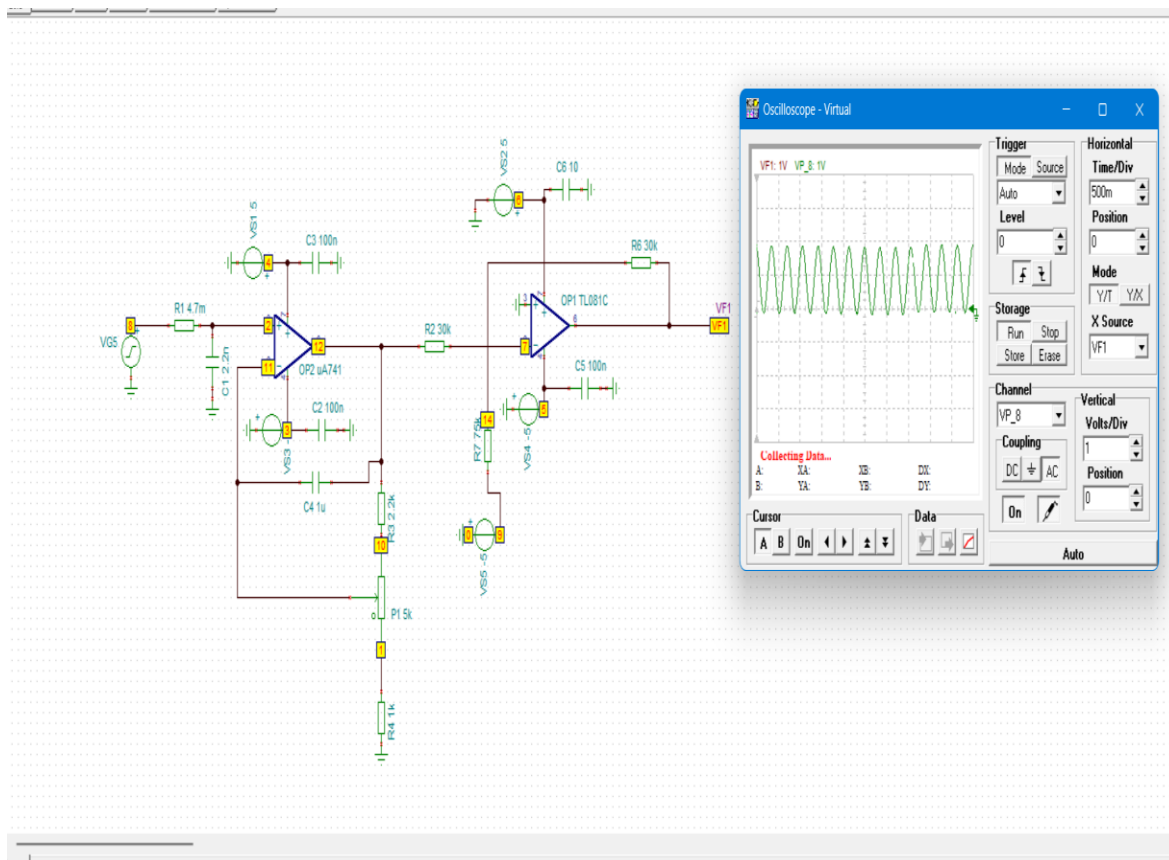


Figure 18: pH TINA simulation

4.4. Entire System Circuit

Developing this schematic was the first step I took to see how all my project's environmental sensors would work together before I printed the PCB. The following diagram shows all these sensors. Each sensor is put in place and connected based on its voltage and communication needs. When the DHT11, TDS, and DS18B20 sensors are directly connected to the 3.3V rail, they work safely at 3.3V. But sensors like the MQ-2, MQ-135, turbidity, and pH sensors need 5V to work right. The design makes this clear by using different power traces.

Figure 19 shows the entire system schematic while figure 20 illustrates the PCB connections.

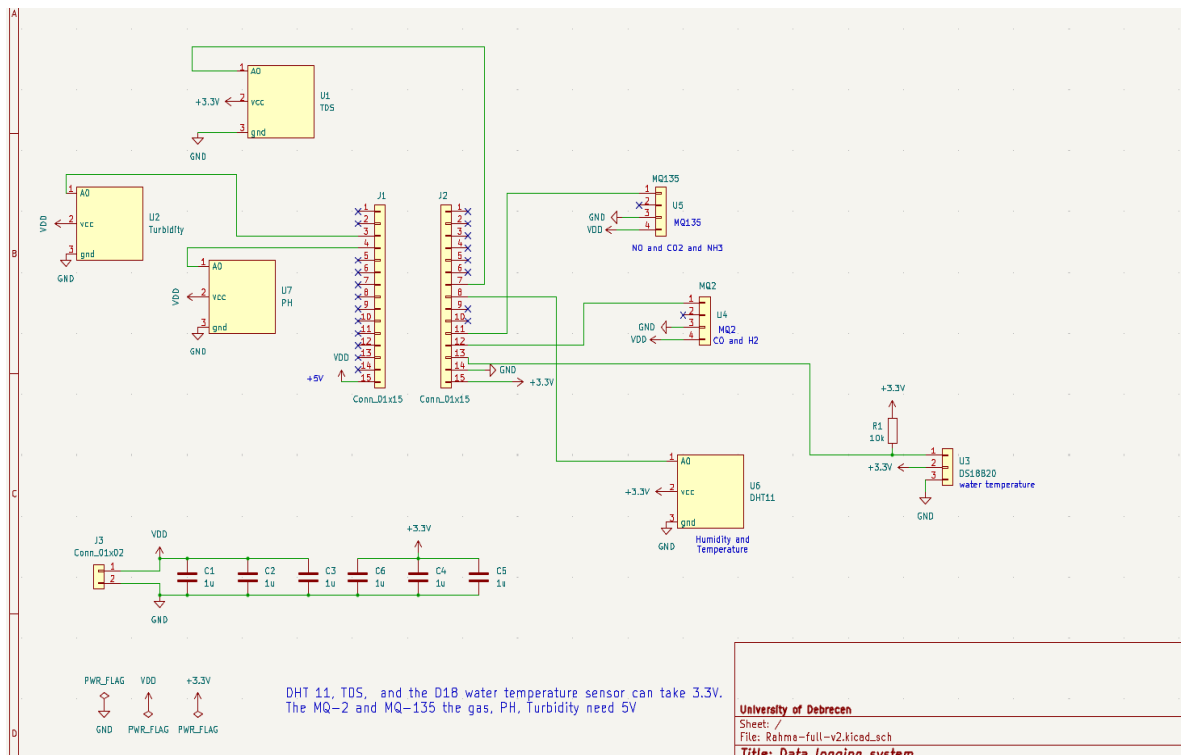


Figure 19: Entire System Schematic

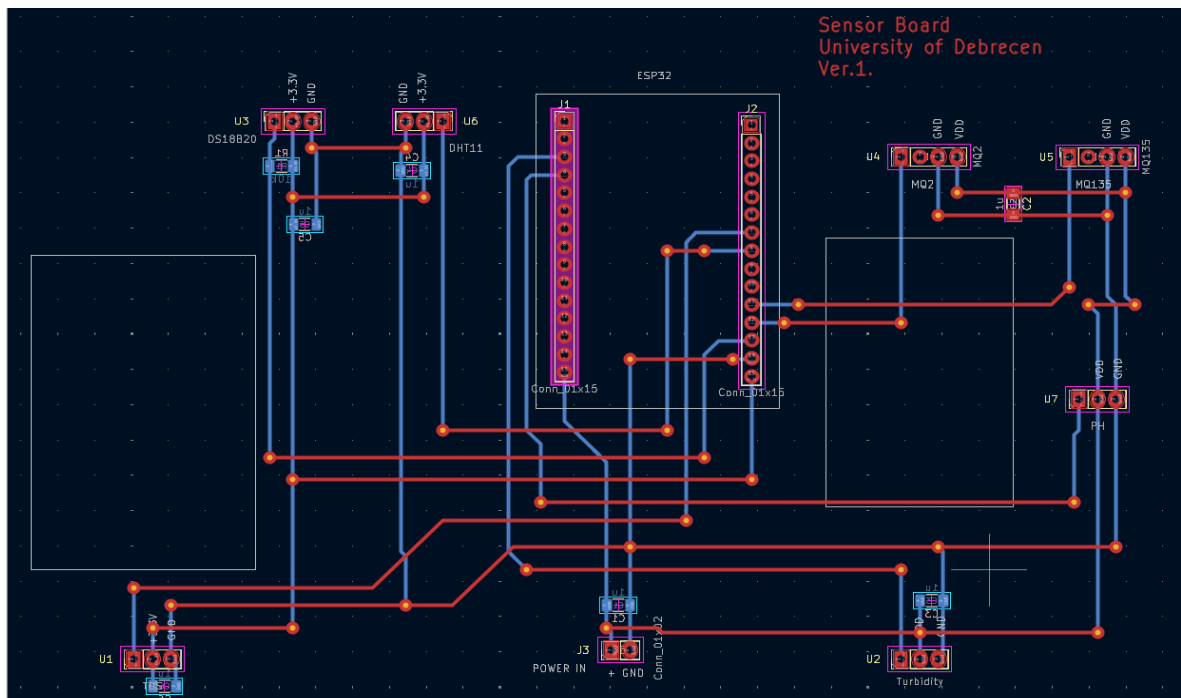


Figure 20: PCB of entire system

4.5. Soldering Components for Entire System

Following the building of the PCB based on the finalized schematic, the soldering process took place to connect the parts permanently and put them together. Due to the voltage needs spelled out in the design, female headers were made to fit the ESP32 microcontroller and each sensor where it should be. To make sure the system had the right amount of voltage, the board had labeled power rails that showed which parts used 3.3V and which used 5V.

Using a digital multimeter to test the continuity and voltage levels at every sensor's connection point helped to guarantee the accuracy of the connections before turning on any sensitive components. This stage was essential in verifying that, based on the requirements of the sensor, every voltage rail provided the proper value—either 3.3V or 5V. The measurements confirmed the PCB didn't have shorts, errors in trace count, or interruptions. Precise instrumentation verification of these connections before full system integration greatly decreased the danger of harming components from unstable voltage or miswiring. Figures 21 and 22 illustrate the top and bottom side of the PCB.

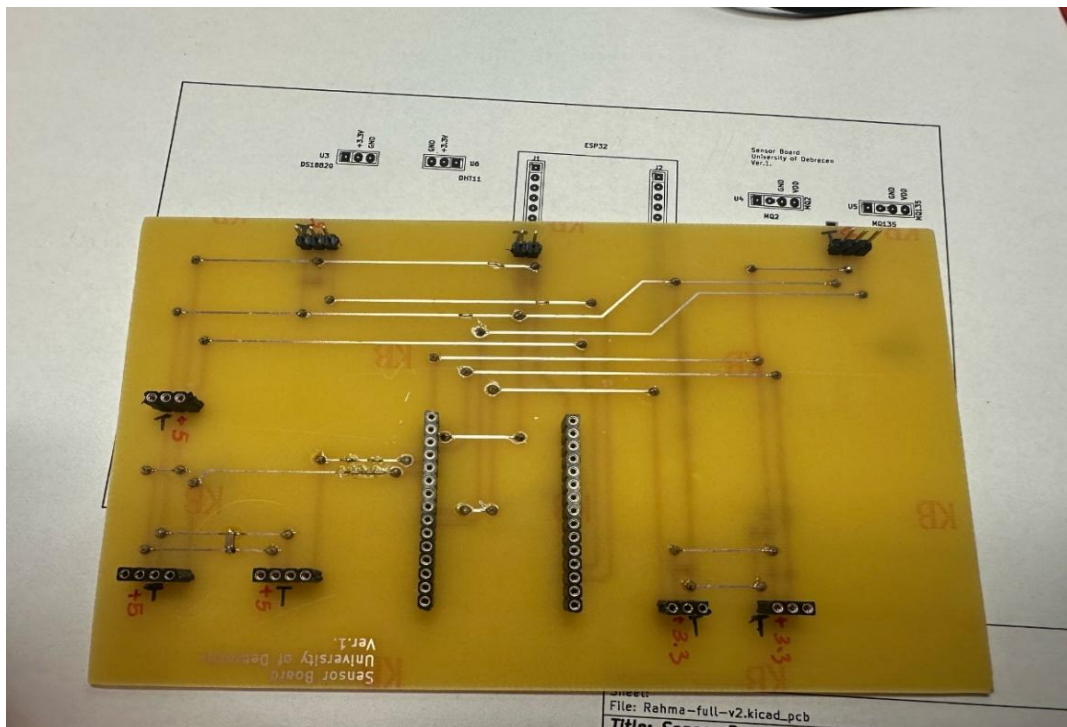


Figure 21: Top side PCB

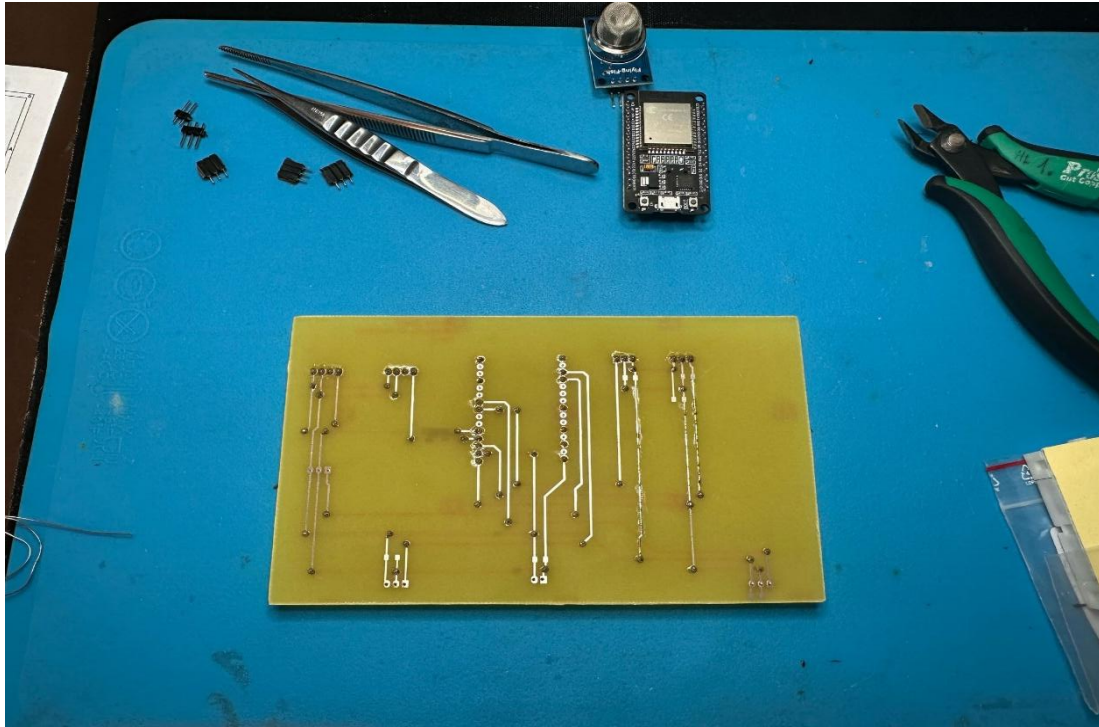


Figure 22: bottom side PCB

4.6. Error Analysis

The MQ-2 and MQ-135 gas sensors presented an obstacle. I first connected them to GPIO 2 and GPIO 4, and they operated perfectly during basic tests. But I discovered the MQ sensors were always displaying zero values once I engaged the ESP32 to Wi-Fi and tried sending the data to ThingSpeak. Digging further I discovered that ADC2 pins cannot be used for analog readings when Wi-Fi is running. This helped clarify why the sensors appeared to stop operating. I moved the MQ-2 and MQ-135 sensors from ADC1 to GPIO 32 and GPIO 33, which remain active even with Wi-Fi turned on. I also changed the PCB wiring to reflect this and following that the MQ sensors began normally reading and uploading to ThingSpeak.

4.7. PCB with Sensors

Figure 23 shows the whole set-up of the system for monitoring the environment. All the sensors are connected to the custom-made PCB that includes the ESP32 microcontroller at its center. The connections and wiring layout are as the schematic design, in respect to the errors that are fixed. This guarantees that the data is sent correctly to the ESP32 to be processed and displayed.

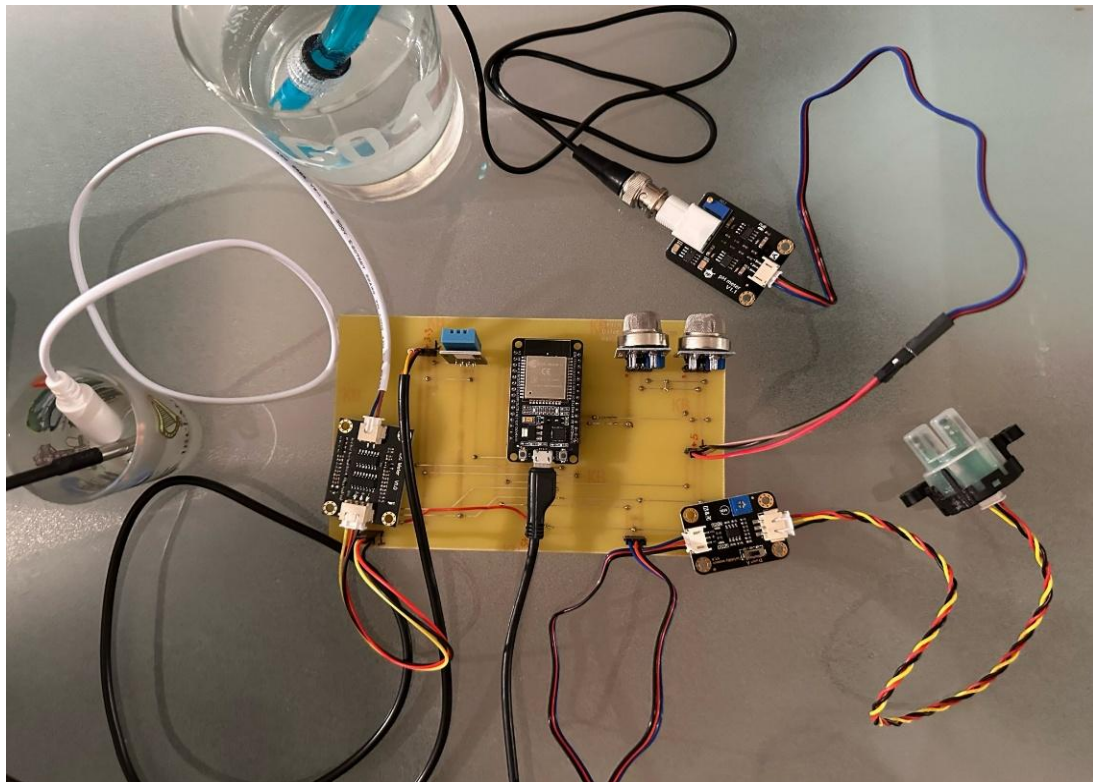


Figure 23: Sensors applied to PCB board

5. Results and Data

Following the hardware's entire assembly and all sensor connectivity to the ESP32, this chapter focuses on the data gathered from the whole environmental monitoring system. Every reading was monitored to verify the sensors were reacting as meant. With data taken directly through serial output, the results include a spectrum of environmental parameters concerning both air and water quality. These tests show whether the system accurately combines several sensors into one board and how well it observes real-time changes. These results were also sent to the ThingSpeak platform to have real time data.

5.1. MQ2 Data

I monitored both carbon monoxide (CO) and hydrogen (H₂) gas levels using the MQ-2 sensor. Stable operation during the tests was made possible by the sensor being connected to GPIO 32 on the ESP32 and powered appropriately from the VIN 5V pin.

From the information recorded on ThingSpeak, I found that over time the H₂ readings varied between 0.5 and 3 ppm while the CO readings ranged between 0.2 and 1.5 ppm. Both gases displayed similar behavior whereby the values began rather low and progressively rose over the course of observation. Particularly at the beginning, the graphs exhibit some natural variations; later, this pattern was smoothed out as the air environment grew rather more active. This trend is typical of gas sensors since they react more clearly in response to minute environmental changes including human presence or minor air movement.

By precisely recording the variations in CO and H₂ concentrations, the MQ-2 sensor showed generally a consistent response. The data gathered is important for environmental monitoring since the slow increase in the measurements over time indicates that the sensor was sensitive enough to identify even minute changes in the surrounding air.

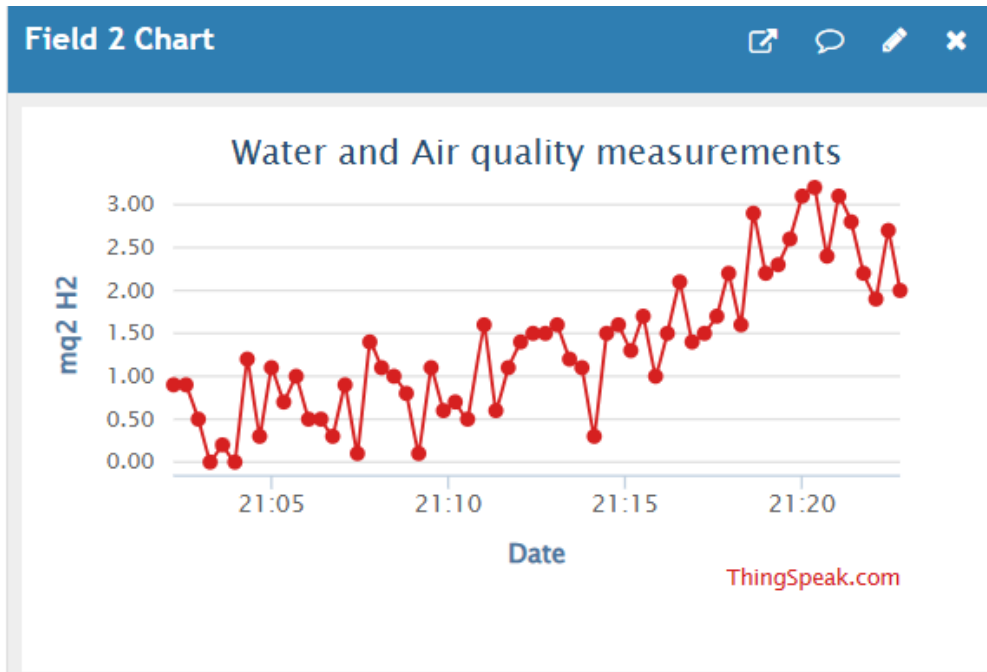


Figure 24: Variation of Hydrogen (H_2) concentration

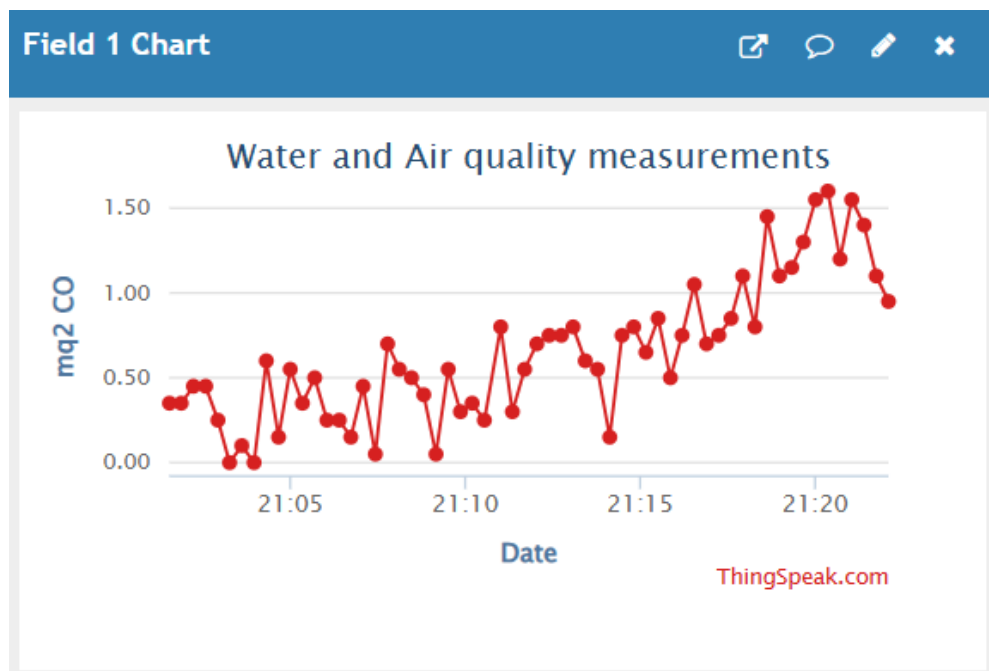


Figure 25: Variation of Carbon Monoxide (CO) concentration

5.2. MQ135 Data

I used the MQ-135 sensor for the measurements of air quality concerning carbon dioxide (CO_2) and ammonia (NH_3) gases.

Examining the data, the CO_2 concentrations ranged from about 3 ppm up to almost 7 ppm over the monitoring period. Over time, CO_2 levels increased gradually and steadily; early on there was some natural variation followed by a more obvious upward rise later in the session. Likewise, by the end of the same period the NH_3 readings began at roughly 2.5 ppm and progressively climbed to roughly 4.5 ppm. Both graphs show that the sensors were able to detect minute environmental changes over time, so reflecting minor changes in gas concentration as conditions varied more around the sensor.

These findings reveal that the MQ-135 sensor could offer real-time valuable information for real-time indoor air quality monitoring.

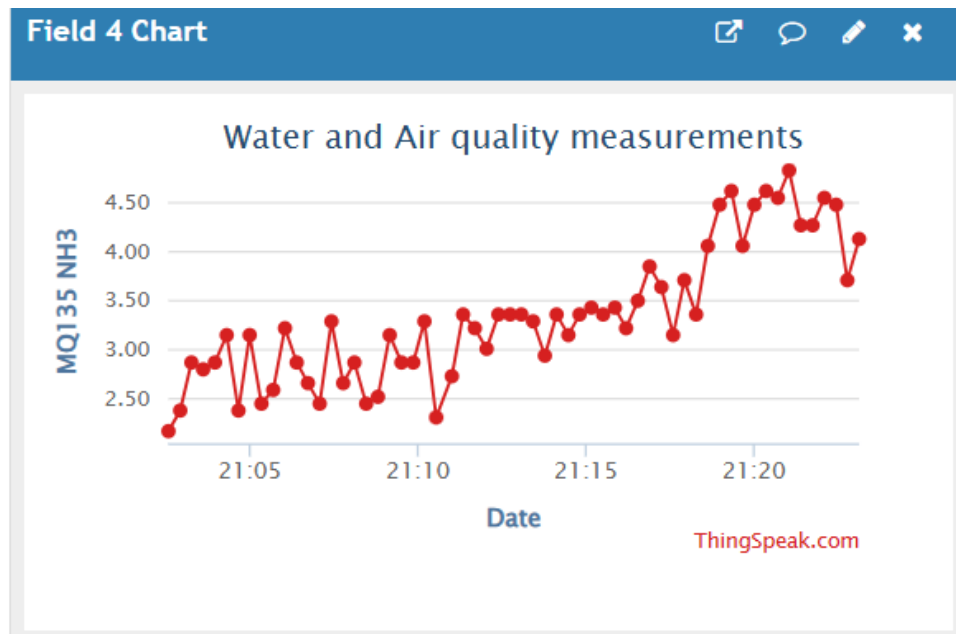


Figure 26: Variation of Ammonia (NH_3) concentration

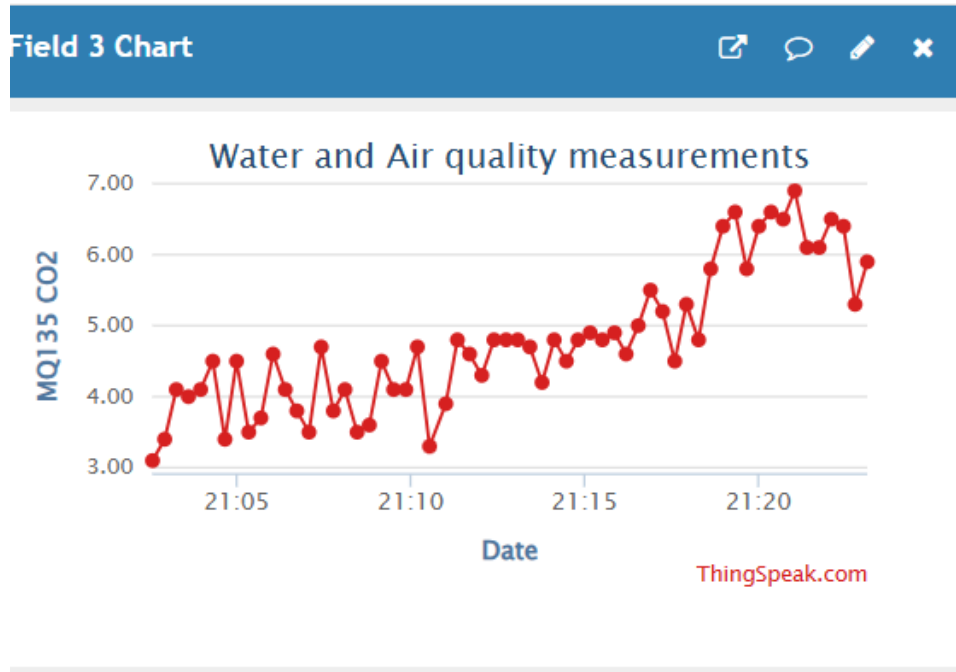


Figure 27: Variation of Carbon Dioxide (CO_2) concentration

5.3. DHT11 Data

The DHT11 sensor tracked the surrounding temperature; the graph shows a rather constant trend with occasional slow changes across time. Starting at about 21.4°C , the temperature increased slowly until it reached about 22.4°C . The middle of the recording showed a small fall, but the values soon recovered and kept rising gradually. These small variations could be the result of changes in air flow surrounding the sensor or natural room temperature shifts, most often brought on by activity close by. DHT11 confirmed that it operated as predicted by constantly responding and offering consistent real-time data, therefore verifying its functionality when included into the system.

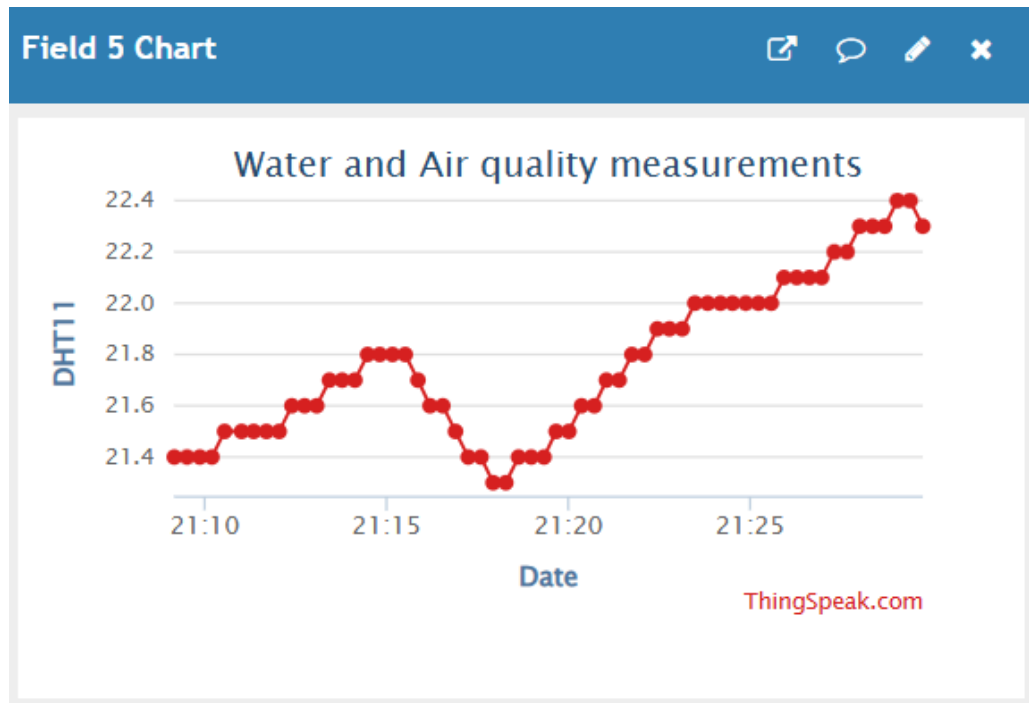
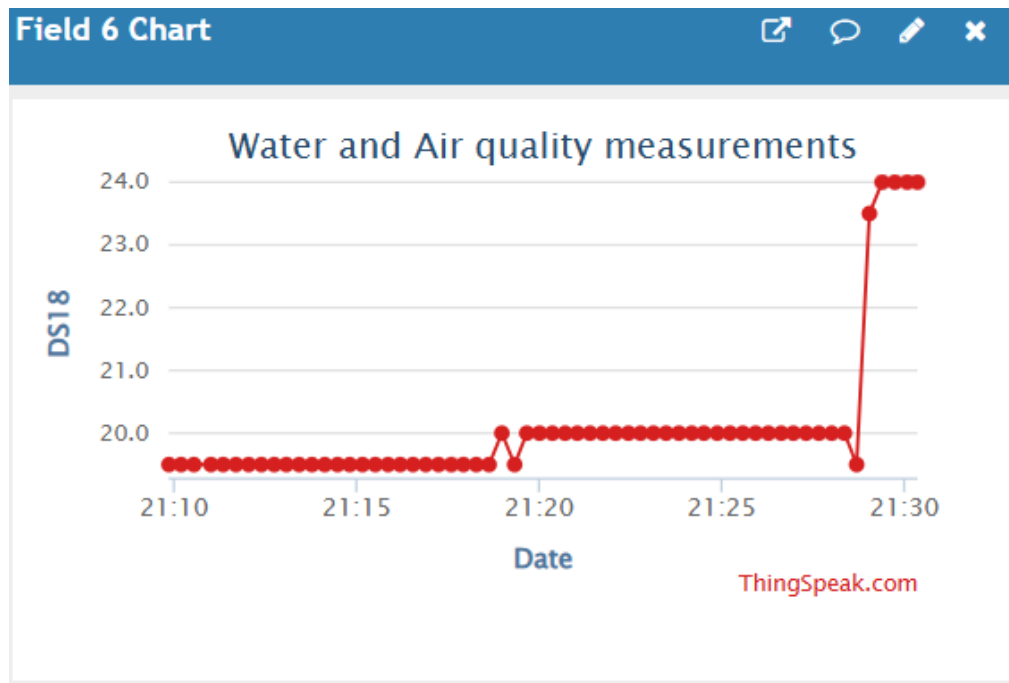


Figure 28: Temperature variation over time

5.4. DS18B20 Data

Between 19.8°C and 20.0°C, the data stayed steady for most of the time. This constant value meant that the water was still and there were no big changes in temperature. But when the time for observation was over, it was clear that the temperature had suddenly gone up to 24.0°C. This change didn't happen by chance; I added warm water on purpose during testing to see how the sensor would react to a change happening in real time. The quick change in the graph shows that the DS18B20 quickly learned the new temperature. For environmental monitoring to work, the sensor needs to be able to respond quickly and accurately to changes in the water. This test helped prove this.



5.5. Turbidity Data

In the beginning, I tested pure water with nothing else added to see how the turbidity sensor responded to changes in the clarity of the water. The results stayed mostly the same at first, between 2.3k and 2.5k, which shows that the water was clean. I added some flour to the water to see how the sensor would respond to a real change. Then the results jumped up to almost 3000, which shows that the sensor did a good job of picking up the sudden rise in particles in suspension.

As the flour got closer to the bottom, the values began to drop again over the next few minutes. Since particles don't all settle down at the same time, there were still some small changes in the data that made sense. This test made it clear to me how sensitive the turbidity sensor is; even a small change, like adding flour, caused the reading to change.

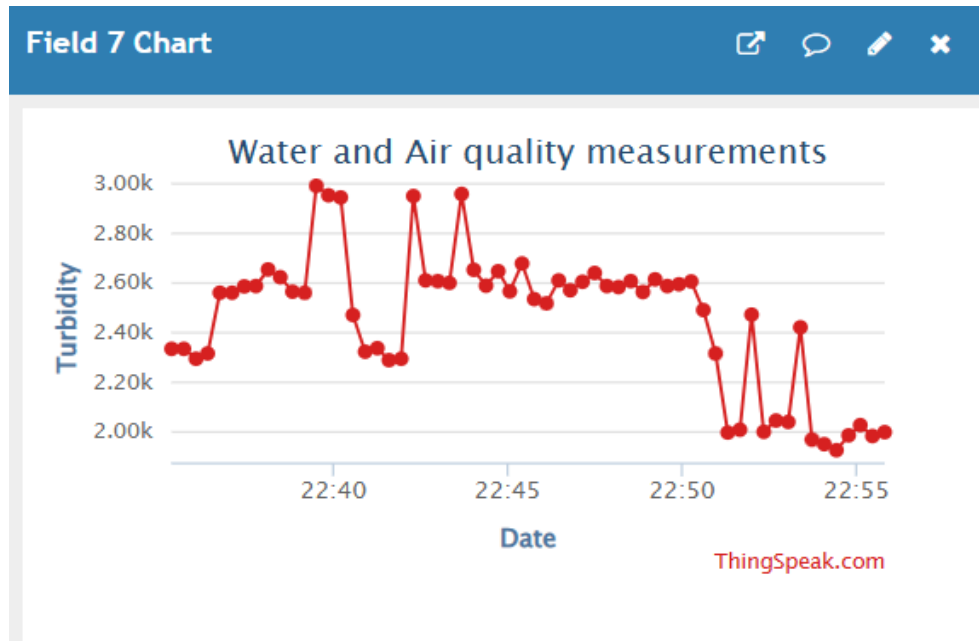


Figure 30: Turbidity sensor readings during testing.

5.6. TDS Data

The TDS values for the first five minutes stayed rather low and steady, averaging about 180 ppm, which is of pure tap water. I mixed a tiny bit of salt in the water to see how the sensor responds to higher solid content. The readings jumped dramatically as soon as I did it, rising to almost 1000 ppm then flattening. This increase suggests that the sensor accurately sensed the spike in conductivity brought on by the dissolved salt. The readings stayed regularly high later, indicating that the salt level of the solution stayed the same. This basic test confirmed that the sensor can constantly pick up changes in water quality in real-time and is operating as predicted.

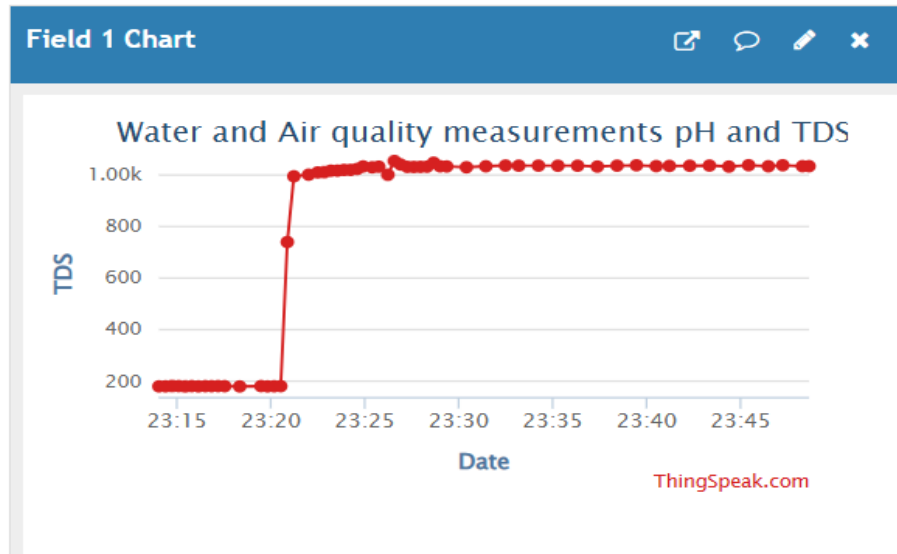


Figure 31: TDS sensor measurements over time.

5.7. PH sensor Data

The graph shows pH sensor data taken from tap water, indicating a constant pH level of about 9.0. I put baking soda into the water; an element known for its alkaline features. The pH values showed a rise, reaching a peak of around 10.2 before stabilizing. This increase matches the expected pH range for baking soda solutions, which often spans from 8 to 10. The data confirms the sensor's performance in detecting differences in water alkalinity and its ability for real-time monitoring of pH shifts.

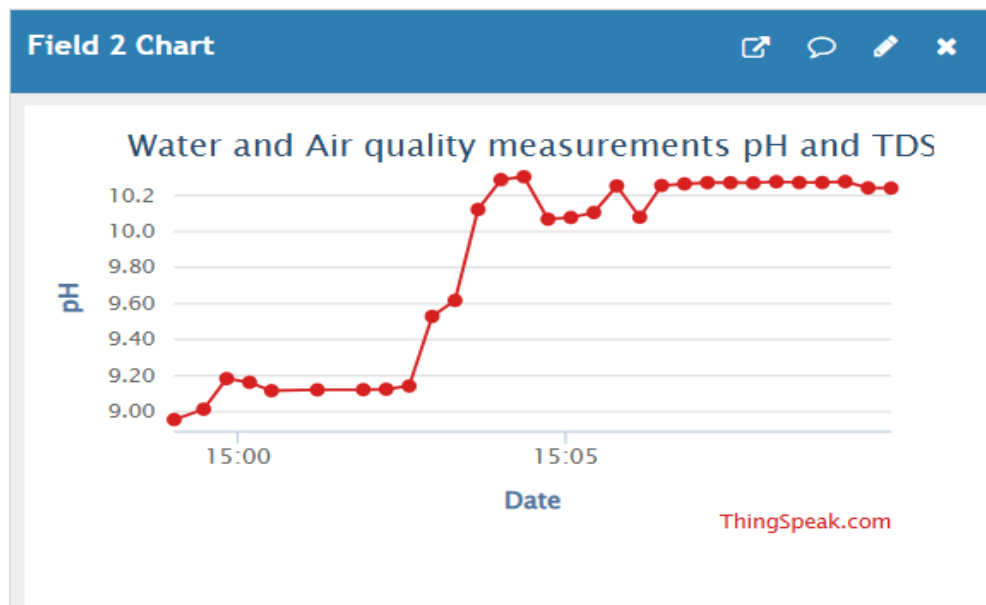


Figure 32: pH sensor measurements over time.

6. Conclusion

I designed and tested a full environmental monitoring system that uses several sensors to check the quality of both air and water. I was able to make a strong system that could collect important environmental data by putting together inexpensive sensors such as the MQ-2 and MQ-135 for gas detection, the DS18B20 for water temperature, the DHT11 for ambient temperature and humidity, and the TDS and turbidity sensors for water analysis. At first, I wanted to make my own pH sensor circuit, but in the end, I chose to use a purchased pH sensor because the readings were more consistent and accurate. I did, however, use an oscilloscope to measure the signal from the custom sensor to make sure it worked and to look at the stages of signal amplification. Moving from the breadboard prototype to a custom PCB layout not only made the system stronger, but it also showed how important it is to have precise pin configuration and voltage regulation for sensors to give accurate readings.

Environmental monitoring is important to find potential pollutants and keep ecosystems healthy, as this project showed. The system was able to send real-time data to the ThingSpeak platform by successfully integrating the ESP32 microcontroller. This lets people analyze and access the data from afar. This feature is helpful for keeping a watch on factories, cleaning water, and checking the air quality, all of which need accurate data to take the right actions. Having to deal with issues like voltage changes, messed up pins, and calibration problems taught me a lot about designing circuits, integrating sensors, and getting data. In the end, the system was able to keep track of environmental factors, showing that it could be used for more types of data analysis and environmental monitoring.

References

- [1] V. Lakshmikantha, A. Hiriyanagowda, A. Manjunath, A. Patted, J. Basavaiah, and A. A. Anthony, "IoT based smart water quality monitoring system," *Global Transitions Proceedings*, vol. 2, no. 2, pp. 181–186, Nov. 2021
- [2] R. K. Nishan *et al*, "Development of an IoT-based multi-level system for real-time water quality monitoring in industrial wastewater," *Discover Water*, vol. 4, (1), pp. 43, 2024
- [3] C. Banciu, A. Florea, and R. Bogdan, "Monitoring and Predicting Air Quality with IoT Devices," *Processes*, vol. 12, no. 9, p. 1961, Sep. 2024,
- [4] J. Bobulski, S. Szymoniak and K. Pasternak, "An IoT System for Air Pollution Monitoring with Safe Data Transmission," *Sensors*, vol. 24, (2), pp. 445, 2024.
- [5] S. Tewari, N. Pandey, and J. Dong, "Air Quality Legislation in Australia and Canada—A Review," *Challenges*, vol. 15, no. 4, p. 43, Dec. 2024,
- [6] O. Alsamrai *et al*, "A Systematic Review for Indoor and Outdoor Air Pollution Monitoring Systems Based on Internet of Things," *Sustainability*, vol. 16, (11), pp. 4353, 2024.
- [7] H. M. Forhad *et al.*, "IoT based real-time water quality monitoring system in water treatment plants (WTPs)," *Heliyon*, vol. 10, no. 23, p. e40746, Nov. 2024,
- [8] Z. Z. Che *et al*, "IoT-Based Water Monitoring Systems: A Systematic Review," *Water*, vol. 14, (22), pp. 3621, 2022.
- [9] J. Ijaradar and S. Chatterjee, "Real-Time Water Quality Monitoring System," 2018.
- [10] K. T. W. A. Primantara, P. Wira Bhuna, and K. Doran, "Water and Air Quality Monitoring System based on the Internet of Things," *Lontar Komputer : Jurnal Ilmiah Teknologi Informasi*, vol. 12, no. 3, p. 151, Nov. 2021.

[11] A. A. Fadhil, Y. A. Fawaz and M. T. Yaseen, "Design and Implementation of Air Pollution Monitoring System in Mosul City," *Instrumentation, Mesure, Metrologie*, vol. 23, (1), pp. 71-77, 2024.

APPENDICES

Appendix 1

```
#include <WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// Wi-Fi credentials
const char* ssid = "TP-Link_2CCC";
const char* password = "87487497";

// ThingSpeak credentials - Channel 1
WiFiClient client;
unsigned long channel1 = 2933030;
const char* writeAPIKey1 = "IF60TQBV6DXP956V";

// ThingSpeak credentials - Channel 2
unsigned long channel2 = 2936986;
const char* writeAPIKey2 = "FFNBWYVDH8UIQAD8";

// Sensor pins
#define MQ2_PIN    32
#define MQ135_PIN  33
#define DHTPIN     5
```

```

#define ONE_WIRE_BUS 15
#define TURBIDITY_PIN 39
#define TDS_PIN 25
#define PH_PIN 34

// DHT11 setup
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// DS18B20 setup
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature ds18b20(&oneWire);

void setup() {
  Serial.begin(115200);
  dht.begin();
  ds18b20.begin();
  analogReadResolution(12); // ESP32 0-4095 ADC

  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");
  ThingSpeak.begin(client);
}

```

```
}
```

```
void loop() {
```

```
  // Read MQ sensor values
```

```
  int mq2Raw = analogRead(MQ2_PIN);
```

```
  int mq135Raw = analogRead(MQ135_PIN);
```

```
  float mq2CO = mq2Raw * 0.05;
```

```
  float mq2H2 = mq2Raw * 0.1;
```

```
  float mq135CO2 = mq135Raw * 0.1;
```

```
  float mq135NH3 = mq135Raw * 0.07;
```

```
  // DHT11 readings
```

```
  float dhtTemp = dht.readTemperature();
```

```
  float dhtHum = dht.readHumidity();
```

```
  // DS18B20 readings
```

```
  ds18b20.requestTemperatures();
```

```
  float waterTemp = ds18b20.getTempCByIndex(0);
```

```
  // Turbidity reading
```

```
  int turbidityRaw = analogRead(TURBIDITY_PIN);
```

```
  float turbidityVoltage = turbidityRaw * (3.3 / 4095.0);
```

```
  float turbidityNTU = (4.2 - turbidityVoltage) * (3000.0 / (4.2 - 2.5));
```

```
  // TDS reading
```

```
  int tdsRaw = analogRead(TDS_PIN);
```

```

float tdsVoltage = tdsRaw * (3.3 / 4095.0);
float tdsPPM = tdsVoltage * 500.0;

// pH reading
int phRaw = analogRead(PH_PIN);
float phVoltage = phRaw * (3.3 / 4095.0);
float phLevel = phVoltage * 3.5; // Adjust the calibration factor as needed

// Print sensor values
Serial.println("---- Sensor Readings ----");
Serial.print("MQ2 CO: "); Serial.print(mq2CO); Serial.println(" ppm");
Serial.print("MQ2 H2: "); Serial.print(mq2H2); Serial.println(" ppm");
Serial.print("MQ135 CO2: "); Serial.print(mq135CO2); Serial.println("
ppm");
Serial.print("MQ135 NH3: "); Serial.print(mq135NH3); Serial.println("
ppm");
Serial.print("DHT11 Temp: "); Serial.print(dhtTemp); Serial.println(" °C");
Serial.print("DS18B20 Water Temp: "); Serial.print(waterTemp); Se-
rial.println(" °C");
Serial.print("Turbidity: "); Serial.print(turbidityNTU); Serial.println("
NTU");
Serial.print("TDS: "); Serial.print(tdsPPM); Serial.println(" ppm");
Serial.print("pH: "); Serial.print(phLevel); Serial.println();

// Upload to ThingSpeak - Channel 1
ThingSpeak.setField(1, mq2CO);
ThingSpeak.setField(2, mq2H2);

```

```
ThingSpeak.setField(3, mq135CO2);
ThingSpeak.setField(4, mq135NH3);
ThingSpeak.setField(5, dhtTemp);
ThingSpeak.setField(6, waterTemp);
ThingSpeak.setField(7, turbidityNTU);
int status1 = ThingSpeak.writeFields(channel1, writeAPIKey1);
Serial.print("ThingSpeak Channel 1 Upload Status: ");
Serial.println(status1);

// Upload to ThingSpeak - Channel 2
ThingSpeak.setField(1, tdsPPM);
ThingSpeak.setField(2, pHLevel);
int status2 = ThingSpeak.writeFields(channel2, writeAPIKey2);
Serial.print("ThingSpeak Channel 2 Upload Status: ");
Serial.println(status2);

delay(20000); // 20 seconds delay (ThingSpeak limit)
}
```